

Program Analysis for Quantitative-reachability Properties

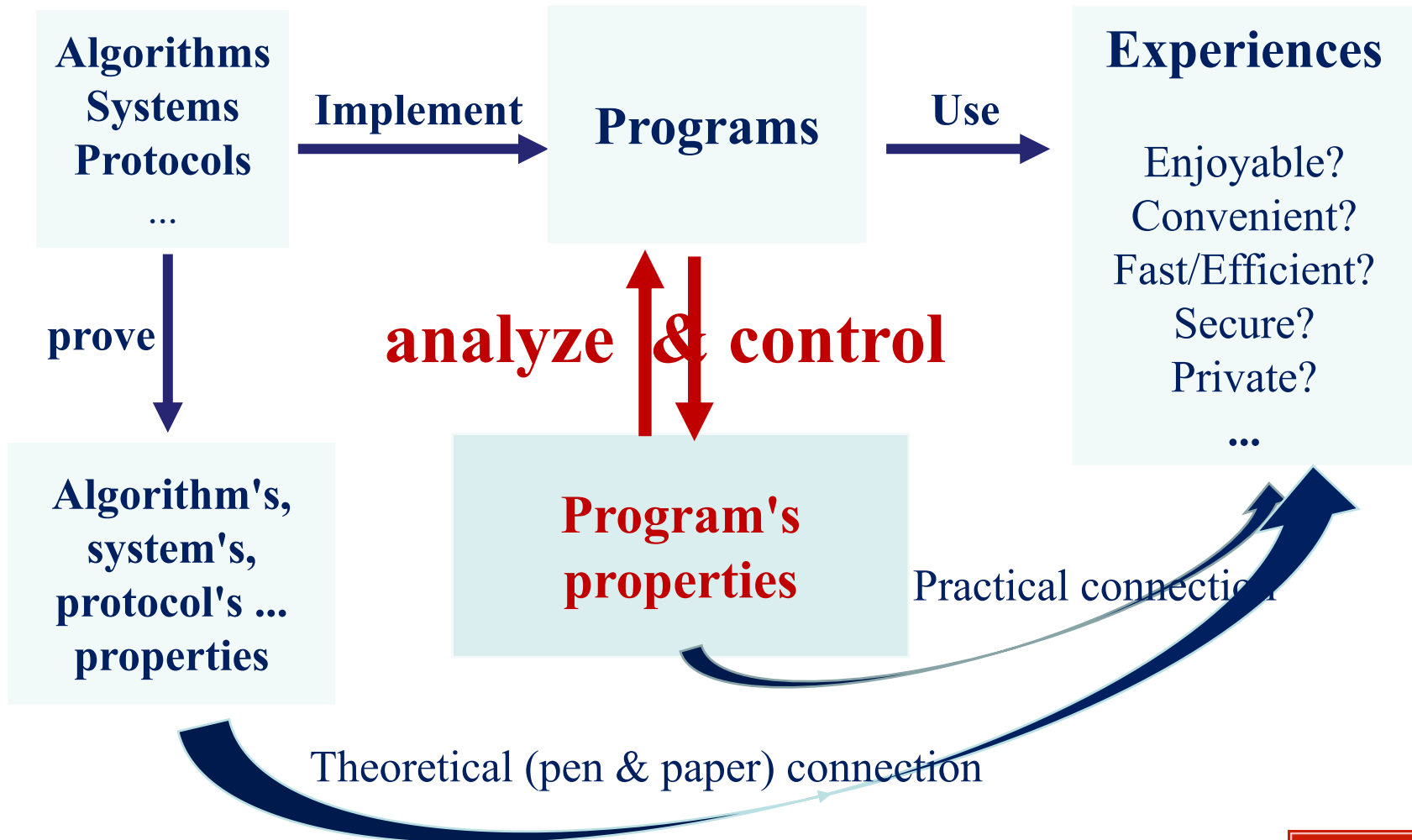
Jiawen Liu

Advised by: Marco Gaboardi

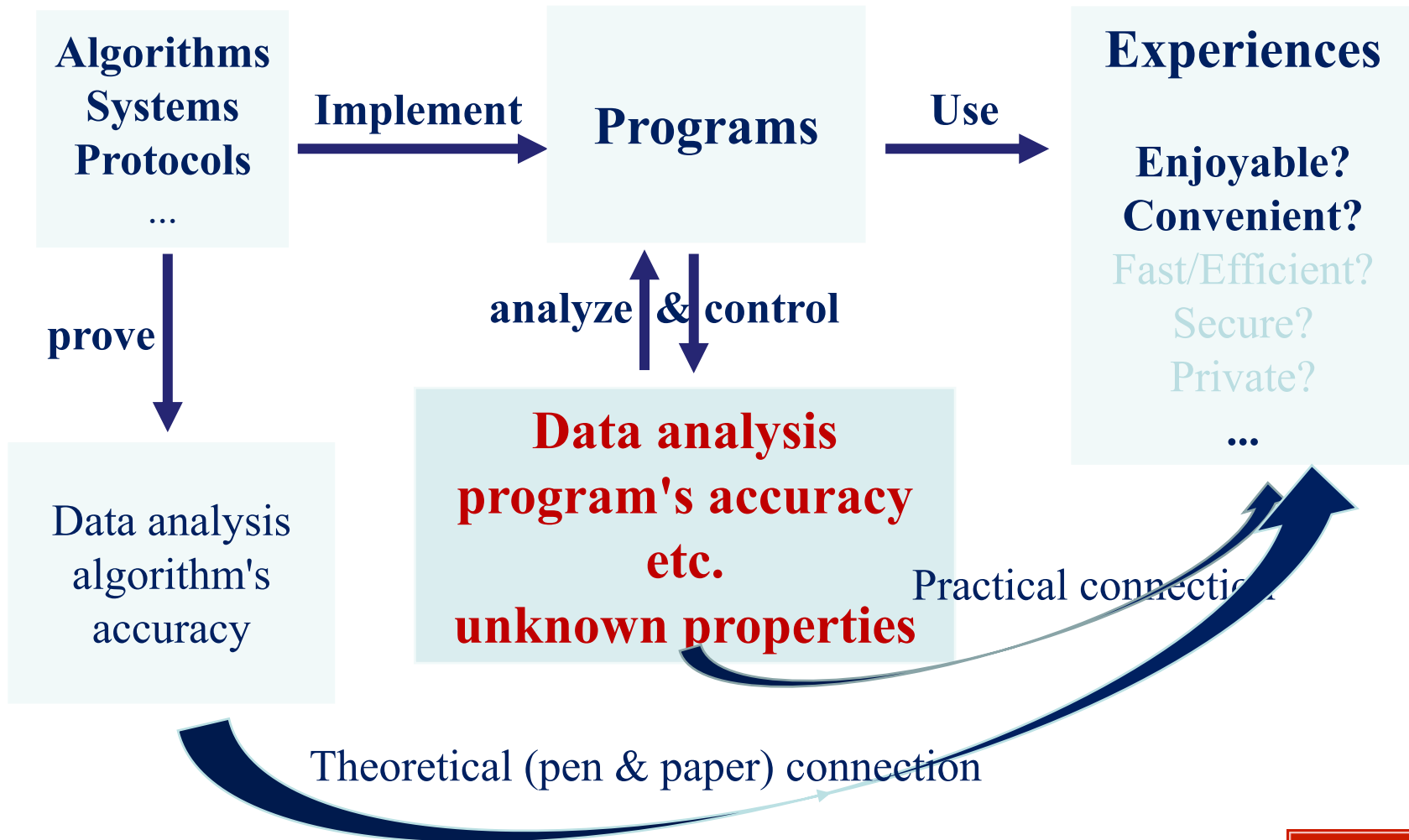
Outline

- Introduction
- Program Analysis for Adaptive Data Analysis
 - Introduction
 - QWhile Language
 - Adaptivity Formalization
 - Program Analysis Algorithm
- Path-sensitive Reachability-bound Analysis (In Submission)
 - Introduction
 - Language Model
 - Reachability-bound
 - Path-sensitive Reachability-bound Algorithm
- Future Works

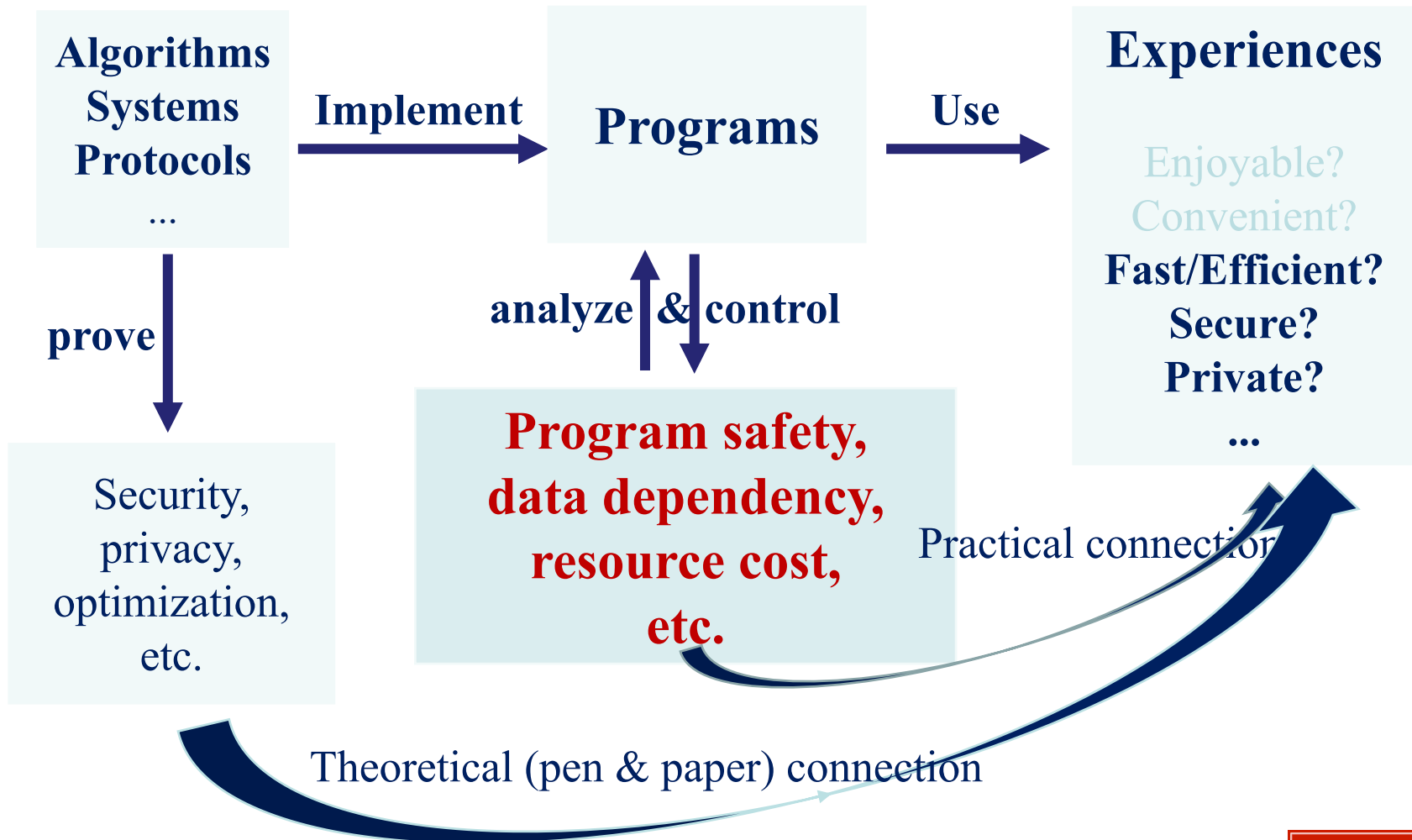
Introduction – program analysis background



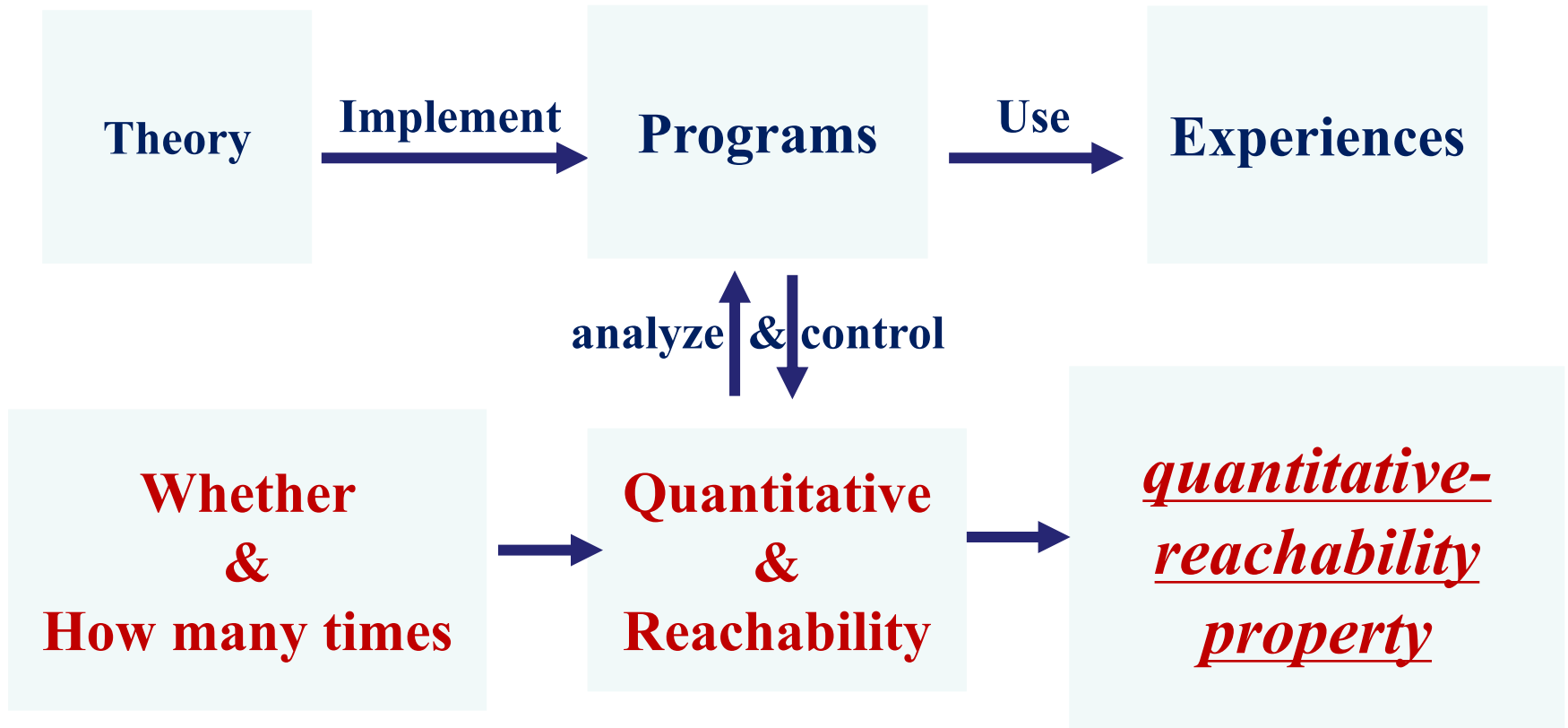
Introduction – program analysis background



Introduction – program analysis background



Introduction – background



Introduction – property formalization challenge

- Reachability properties:
 - Variable/Data/Code reachability, commutative, liveness, etc.**
 - whether** code is executed, data is reachable / live / used / depends on others
- Quantitative properties:
 - Resource cost / Worst-case execution time / Complexity, etc.**
- *What is quantitative-reachability* hyper-property?

Introduction – new methodology

Semantics-based quantitative-reachability formalization

- Define sub-property "*quantitative*" & "*reachability*",
- ➔
- Define new *structure/model* carrying the two sub-properties,
- ➔
- Define the "*quantitative-reachability*" as a hyper-property over the model

Introduction – program analysis challenge

- Reachability analysis techniques:

Control-flow, data-flow, liveness, dependency analysis^[1], etc.

→ **whether** code is executed, data is reachable / live / used / depends on others

→ only "*reachability*"

- Quantitative analysis techniques:

Resource cost, worst-case execution time, complexity estimation

→ **overall** execution time, resource cost, complexity, etc.

→ only "*quantity*"

- *Quantitative-reachability* analysis techniques:

Reachability-bound analysis^[2], etc.

→ **estimate only the "*quantity*", and not precise enough**

[1] Patrick Cousot. 2019. Abstract Semantic Dependency. SAS'19

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Introduction – property analysis challenge

- *Trivially applying two analyses:*

analyzing safety, data dependency, resource usage by *applying two techniques*.

- Reachability Analysis

→ Whether a program leaks a secret / use some data

- Quantitative Analysis

→ Overall information leakage / resource usage / execution time

→ *Disconnection :*

- *Two pieces properties aren't associate to each other*
quantity $\propto$ reachability

Introduction – new methodology

➔ Analyze the quantitative-reachability integrally:

Analyze sub-properties "quantitative" & "reachability"

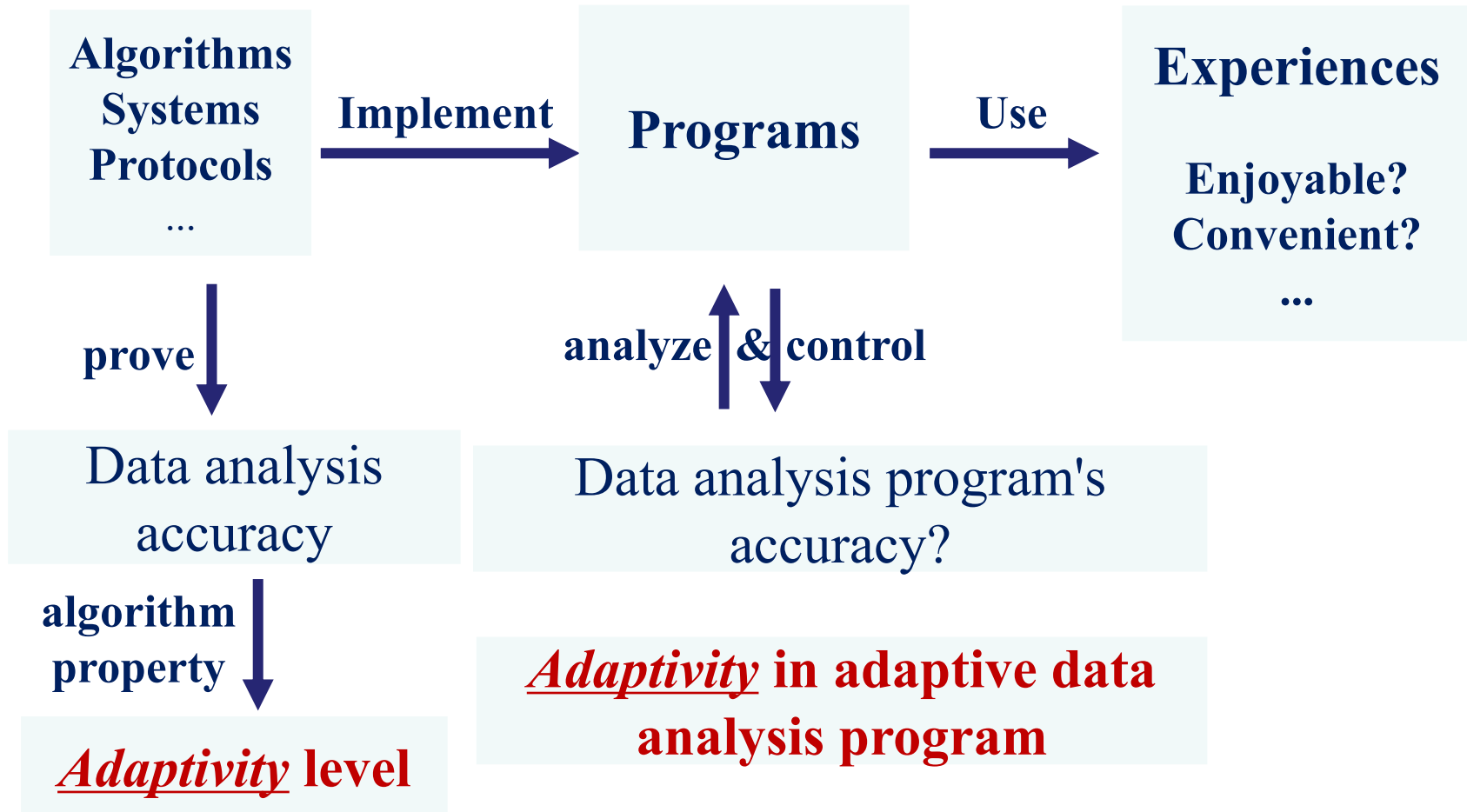


Estimate the new structure/model

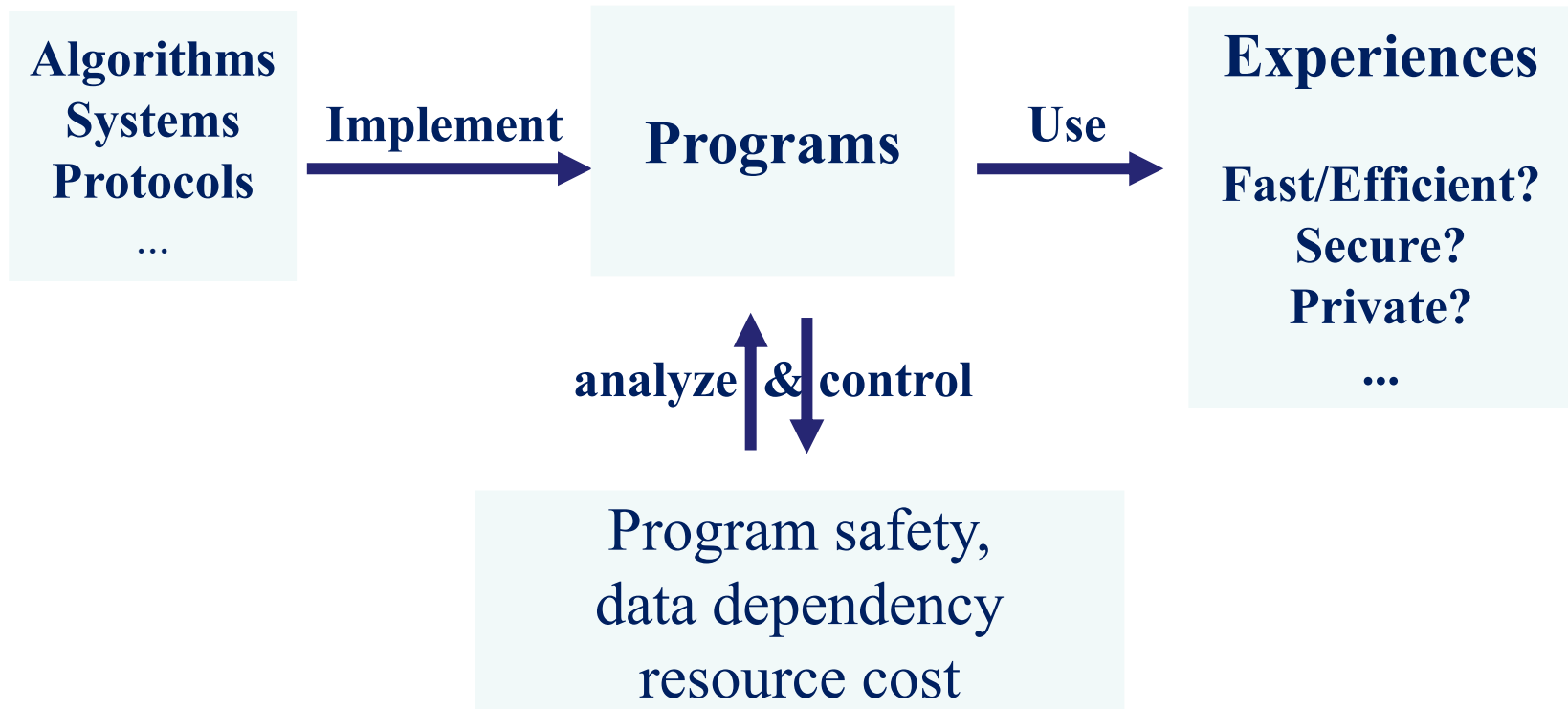


Analyze the hyper-property "quantitative-reachability"

Introduction – two quantitative-reachability properties



Introduction – two quantitative-reachability properties



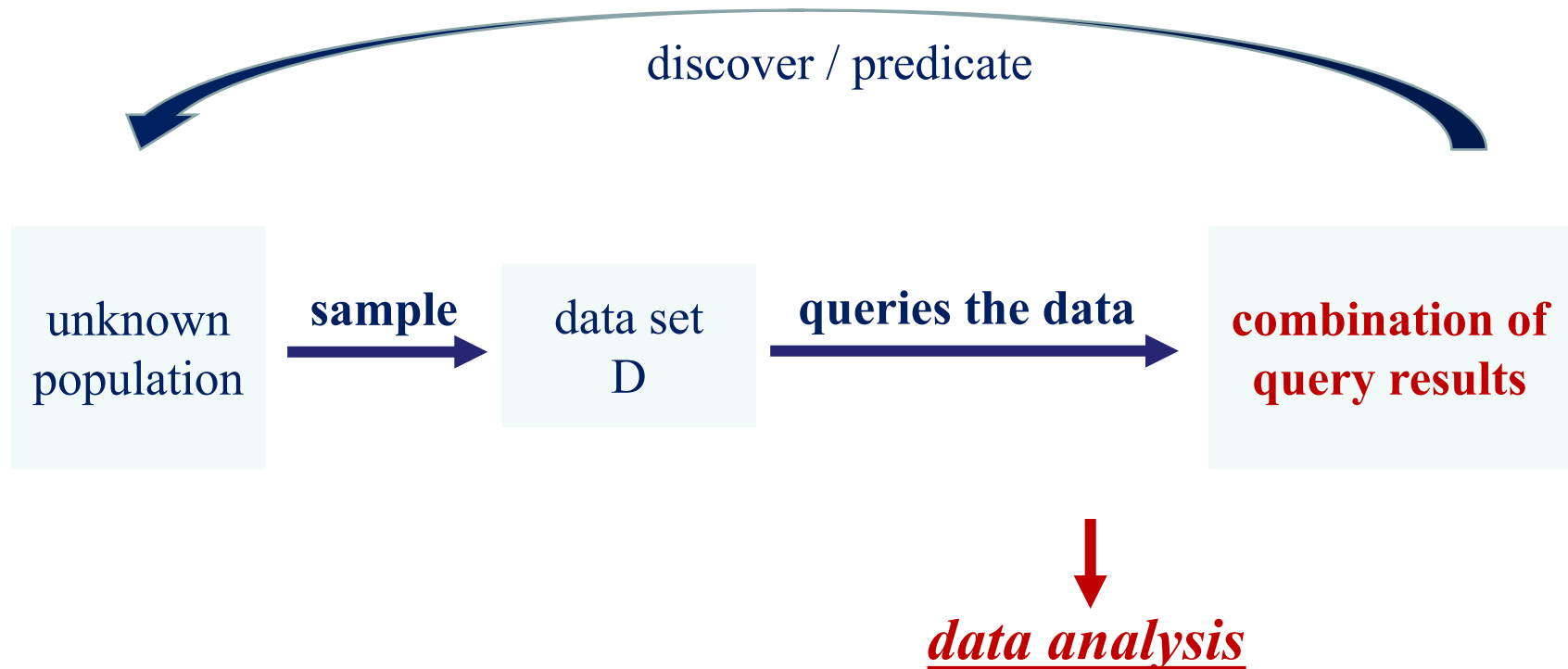
The visiting times of each program location during the program execution

PART I :

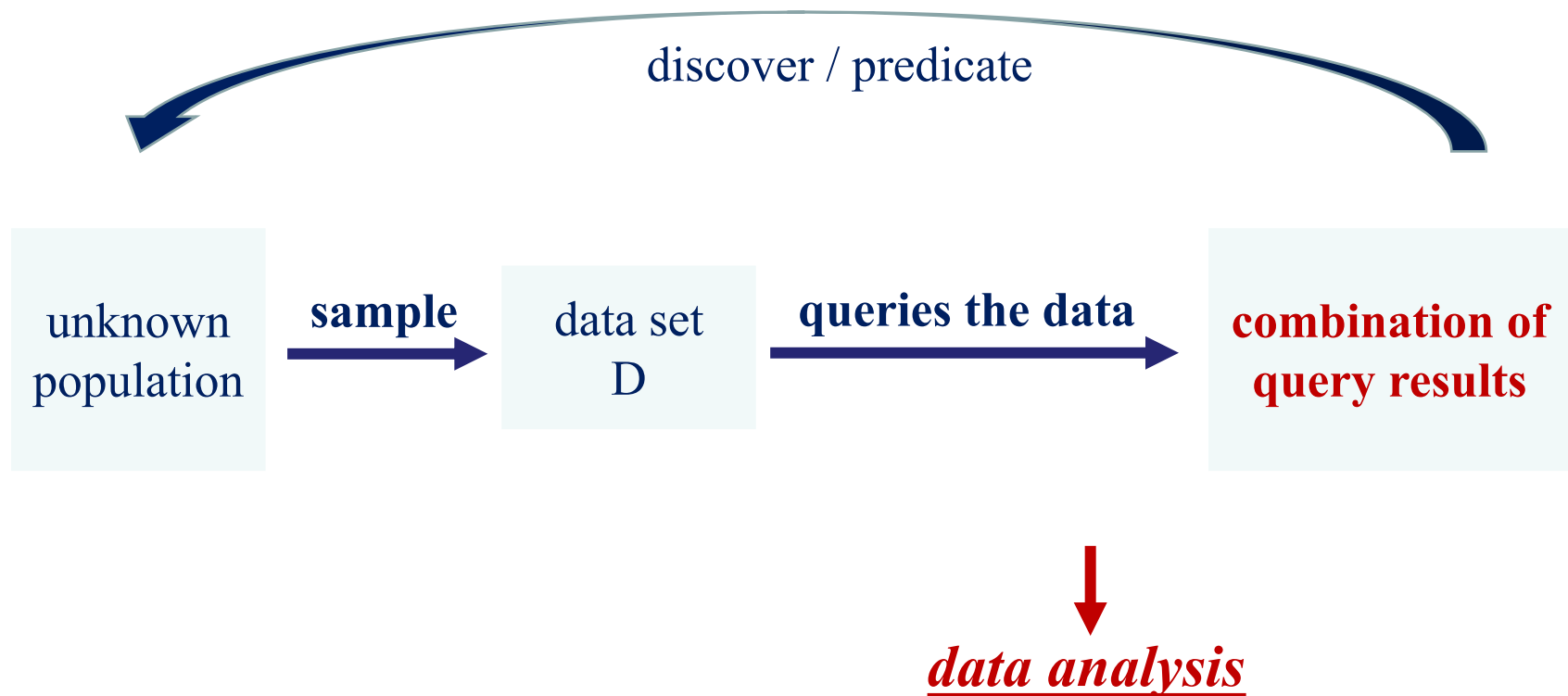
Program Analysis for Adaptive Data Analysis

- Introduction:
 - Adaptivity
 - Motivation
- QWhile Language
- Adaptivity Definition
- Adaptivity Estimation Algorithm

Introduction – adaptive data analysis



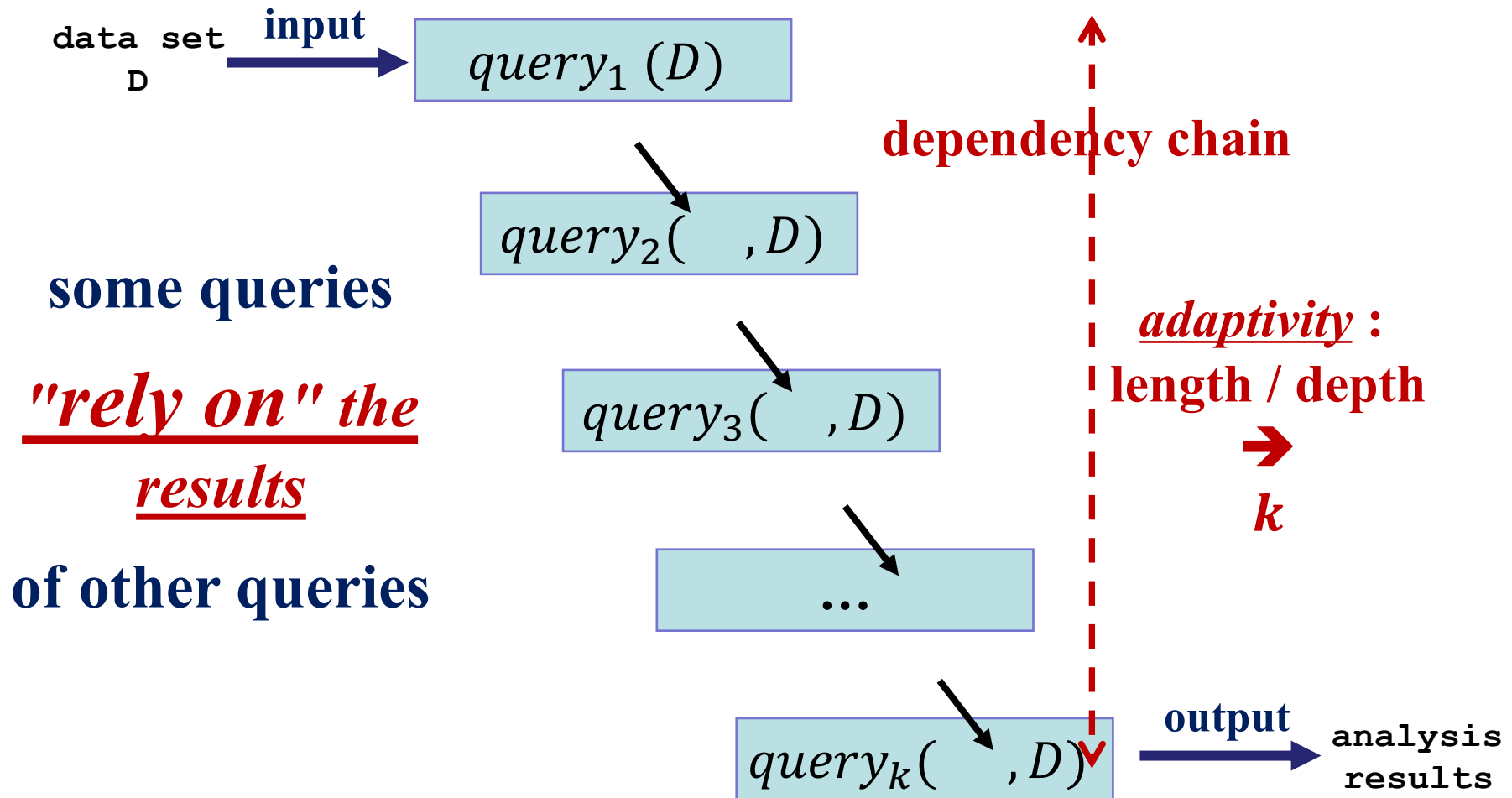
Introduction – adaptive data analysis



example program

```
multiRounds(k) :=  
  [a ← 0] 0;  
  [j ← k] 1;  
  while [(j > 0)] 2 do (  
    [a ← query( $\mathcal{X}[a]$ )] 3;  
    [j ← j - 1] 4  
  )
```

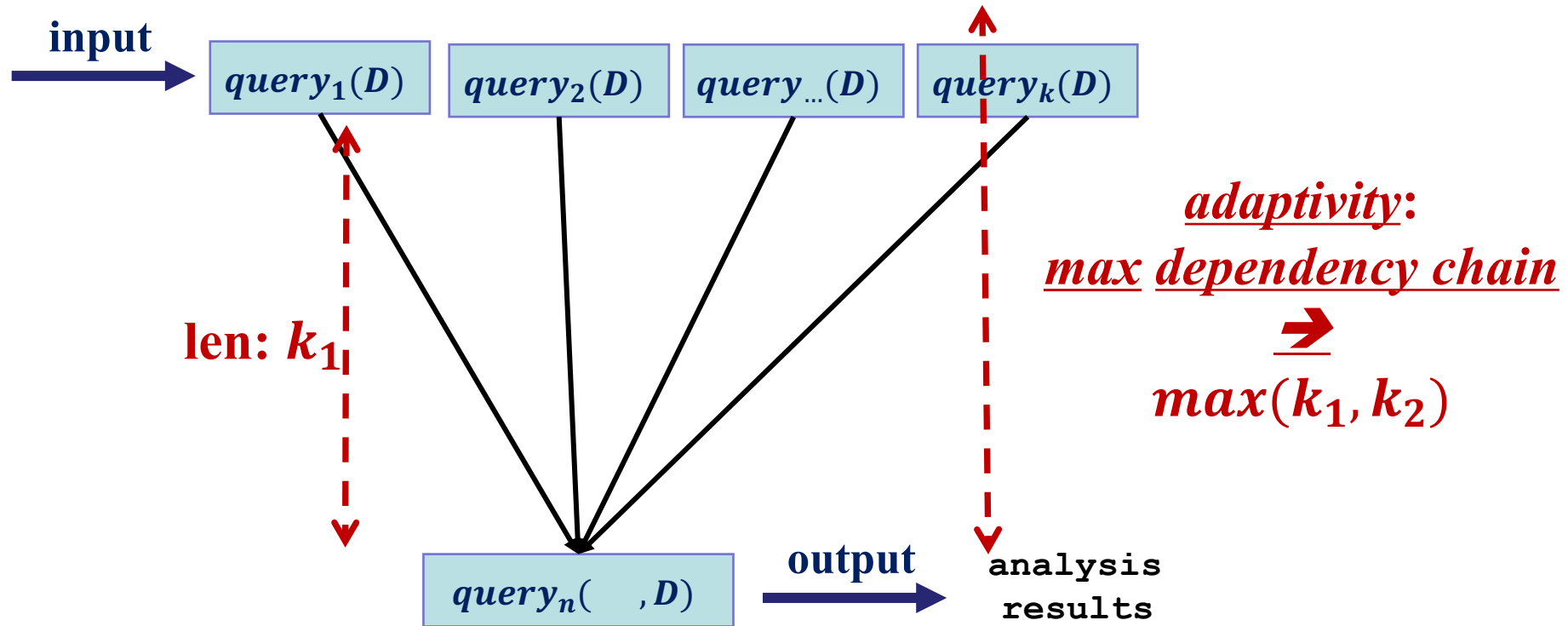
Introduction – adaptivity in adaptive data analysis



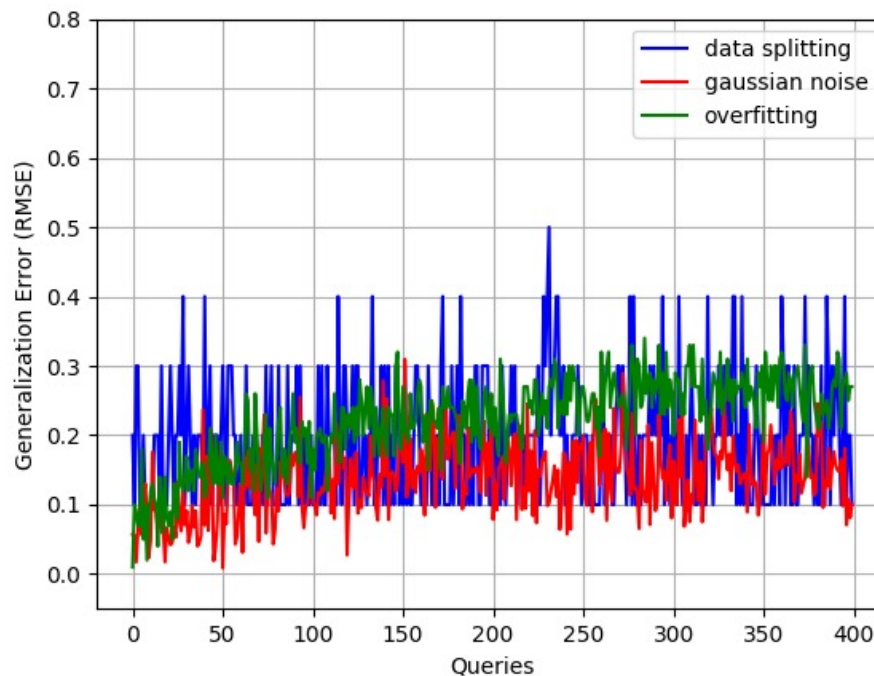
QWhile language – example program

```
twoRounds(k) :=  
  [a ← 0] 0;  
  [j ← k] 1;  
  while [(j > 0)] 2 do (  
    [x ← query( $\mathcal{X}[j]$ )] 3;  
    [j ← j - 1] 4;  
    [a ← a + x] 5);  
  [1 ← query( $\mathcal{X}[k] * a$ )] 6
```

Introduction – adaptivity in adaptive data analysis

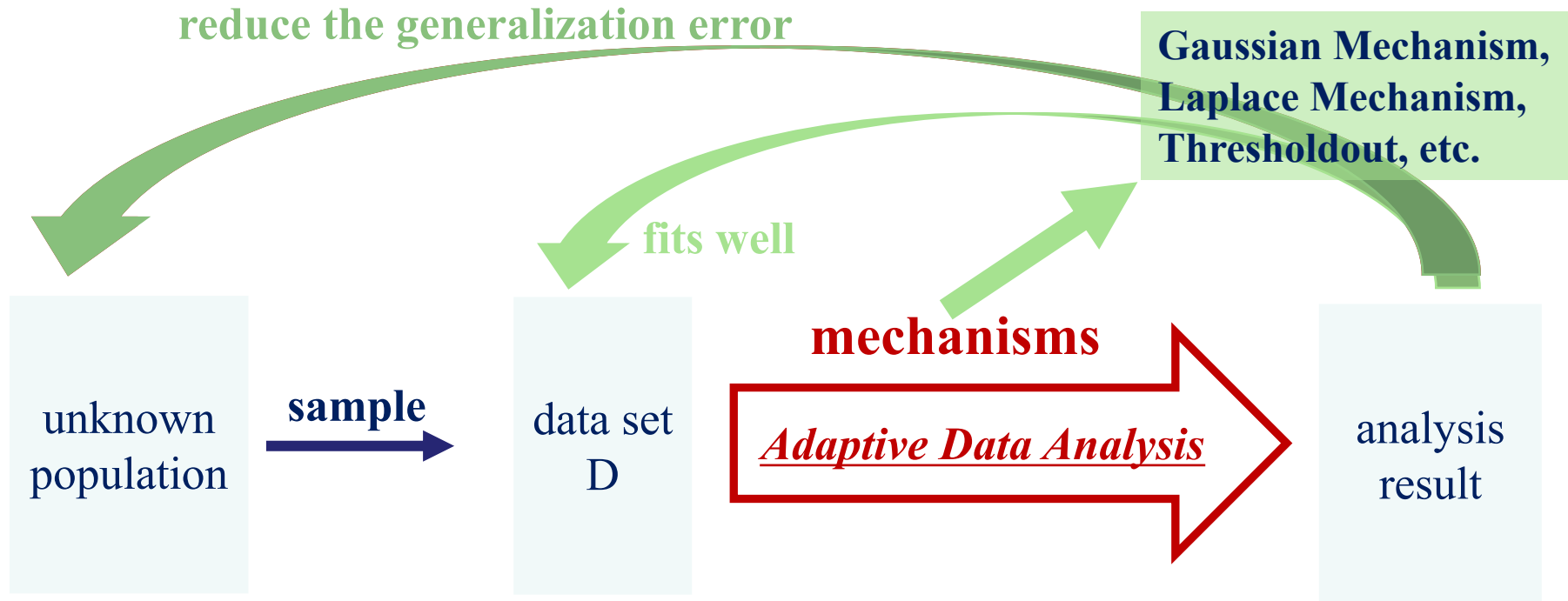


Introduction – control the generalization error theoretically



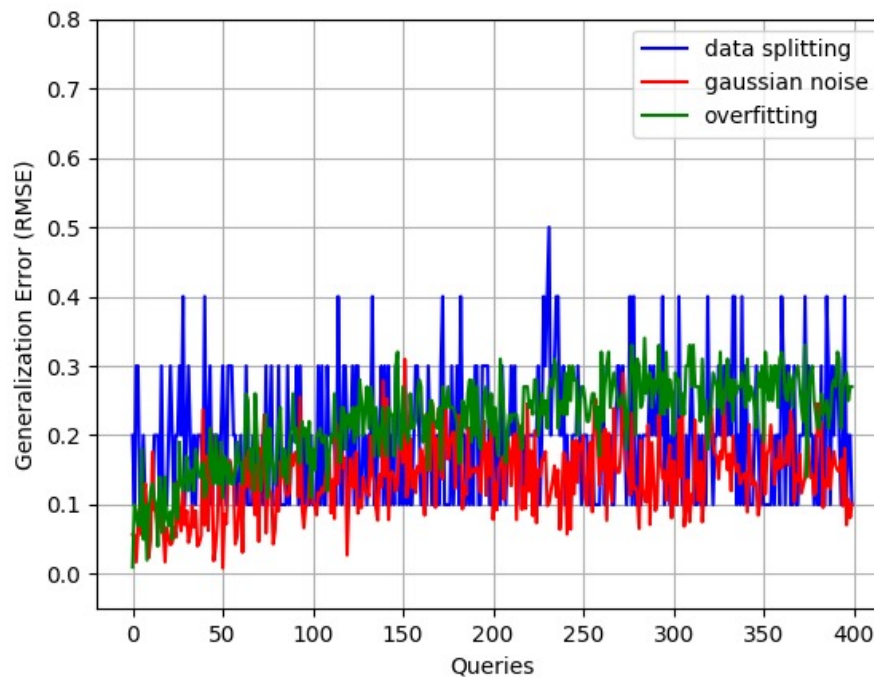
the "adaptivity" in a data analysis propagates the error

Introduction – control the generalization error theoretically



use "adaptivity" to choose different mechanisms,
to reduce the generalization error

Introduction – control the generalization error theoretically



the "adaptivity" in a data analysis propagates the error

Research challenges & goals

- Challenge – I : program model for adaptive data analysis
 - QWhile language

- Challenge – II : adaptivity formalization
 - New adaptivity definition

- Challenge – III : adaptivity estimation
 - Program analysis algorithm for adaptivity

Adaptivity Analysis Framework

QWhile language

Expression	$e ::= v \mid a \mid b \mid [e, \dots, e]$
Value	$v ::= n \mid \text{true} \mid \text{false} \mid [] \mid [v, \dots, v]$
Query Expression	$\psi ::= \alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$
Query Value	$\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$
Labeled Command	$c ::= [x \leftarrow e]^l \mid [x \leftarrow \text{query}(\psi)]^l \mid \text{while } [b]^l \text{ do } c$ $\mid c; c \mid \text{if } ([b]^l, c, c) \mid [\text{skip}]^l$

QWhile language – query expression

Query Expression $\psi ::= \alpha \mid a \mid \psi \oplus_a \psi \mid \chi[a]$

Query Value $\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$

- $\chi[n]$: access the values at *n-th column* of a *hidden* database
- $\text{query}(\chi[n])$: the *average* value of $\chi[n]$ over each row of the database

$$\text{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=0}^n \chi_i[3] + 5$$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 1. Normalize the query expression: $\langle \tau, \psi \rangle \Downarrow_q \alpha$.

The query expression ψ is first simplified to its normal form α

$$\alpha ::= n \mid \chi[n] \mid \alpha \oplus_a \alpha \mid n \oplus_a \chi[n] \mid \chi[n] \oplus_a n$$

$$\frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, a \rangle \Downarrow_q n} \quad \frac{\langle \tau, \psi_1 \rangle \Downarrow_q \alpha_1 \quad \langle \tau, \psi_2 \rangle \Downarrow_q \alpha_2}{\langle \tau, \psi_1 \oplus_a \psi_2 \rangle \Downarrow_q \alpha_1 \oplus_a \alpha_2} \quad \frac{\langle \tau, a \rangle \Downarrow_a n}{\langle \tau, \chi[a] \rangle \Downarrow_q \chi[n]} \quad \frac{}{\langle \tau, \alpha \rangle \Downarrow_q \alpha}$$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 2. Compute query request and get the result: $\text{query}(\alpha) = v$

Expected value of the function $\lambda \chi . \alpha$ applied to each row of the hidden database:

$$\text{query}(\chi[3] + 5) = \frac{1}{n} \sum_{i=0}^n \chi_i[3] + 5$$

Summarize the computation procedure into $\text{query}(\alpha) = v$

QWhile language – trace-based semantics

- Query command evaluation

$$\frac{\langle \tau, \psi \rangle \Downarrow_q \alpha \quad \text{query}(\alpha) = v \quad \epsilon = (x, l, v, \alpha)}{\langle [x \leftarrow \text{query}(\psi)]^l, \tau \rangle \rightarrow \langle [\text{skip}]^l, \tau :: \epsilon \rangle} \text{query}$$

- 3. Generate the event for query assignment : $\epsilon = (x, l, v, \alpha)$

x : the variable name

l : the label for this query command

v : the query result computed over unknown database

α : the normalized the query request

Adaptivity Formalization

- ➔ "Reachability" aspect:
Data dependency relation
- ➔ "Quantitative" aspect:
Times that each command is evaluated
- ➔ "Structure" carrying the two sub-properties
Semantics-based dependency graph
- ➔ "Adaptivity" as a quantitative-reachability hyper-property

→ "Reachability" as **dependency relation**^[1]

- Variable may-dependency:

$$\text{DEP}(x^i, y^j, c) \in \{\text{true}, \text{false}\}$$

- **witness :**

- τ_0 initial trace, ϵ_1 for x^i and ϵ_2 for y^j

- execution 1:

$$\langle c, \tau_0 \rangle \rightarrow^* \langle c_1, \tau_{1++}[\epsilon_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon_1]_{++} \tau_{++}[\epsilon_2] \rangle$$

- execution 2:

$$\langle c_1, \tau_{1++}[\epsilon'_1] \rangle \rightarrow^* \langle c_2, \tau_{1++}[\epsilon'_1]_{++} \tau'_{++}[\epsilon'_2] \rangle$$

observe value in ϵ_2 is changed or ϵ_2 is disappeared

$$\epsilon_2 = (x, l, v, \alpha) \text{ v.s. } \epsilon'_2 = (x, l, v, \alpha')$$

[1] Patrick Cousot. 2019. Abstract Semantic Dependency. SAS'19

Example – reachability as **data dependency relation**

```
twoRounds(k) :=  
  [a ← 0] 0;  
  [j ← k] 1;  
  while [(j > 0)] 2 do (  
    [x ← query( $\mathcal{X}[j]$ )] 3;  
    [j ← j - 1] 4;  
    [a ← a + x] 5);  
  [l ← query( $\mathcal{X}[k] * a$ )] 6
```

- $DEP(x^3, a^5, c), DEP(a^5, l^6, c)$

➔ "Quantitative" as the weight of a labeled variable^[1,2]

- Weight of labeled variable x^l in program c :

$$w: \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$$

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Example – quantitative as evaluation **times**

```

twoRounds(k)  :=
  [a ← 0] 0;
  [j ← k] 1;
  while [(j > 0)] 2 do (
    [x ← query( $\mathcal{X}[j]$ )] 3;
    [j ← j - 1] 4;
    [a ← a + x] 5);
  [1 ← query( $\mathcal{X}[k] * a$ )] 6

```

- for a^0 : $w = \lambda \tau_0.1$,
- for x^3 : $w = \lambda \tau_0.\max(0, \rho(\tau_0, k))$

➔ New "structure" as semantics-based dependency graph

$$G(c) = (V(c), E(c), W(c), Q(c))$$

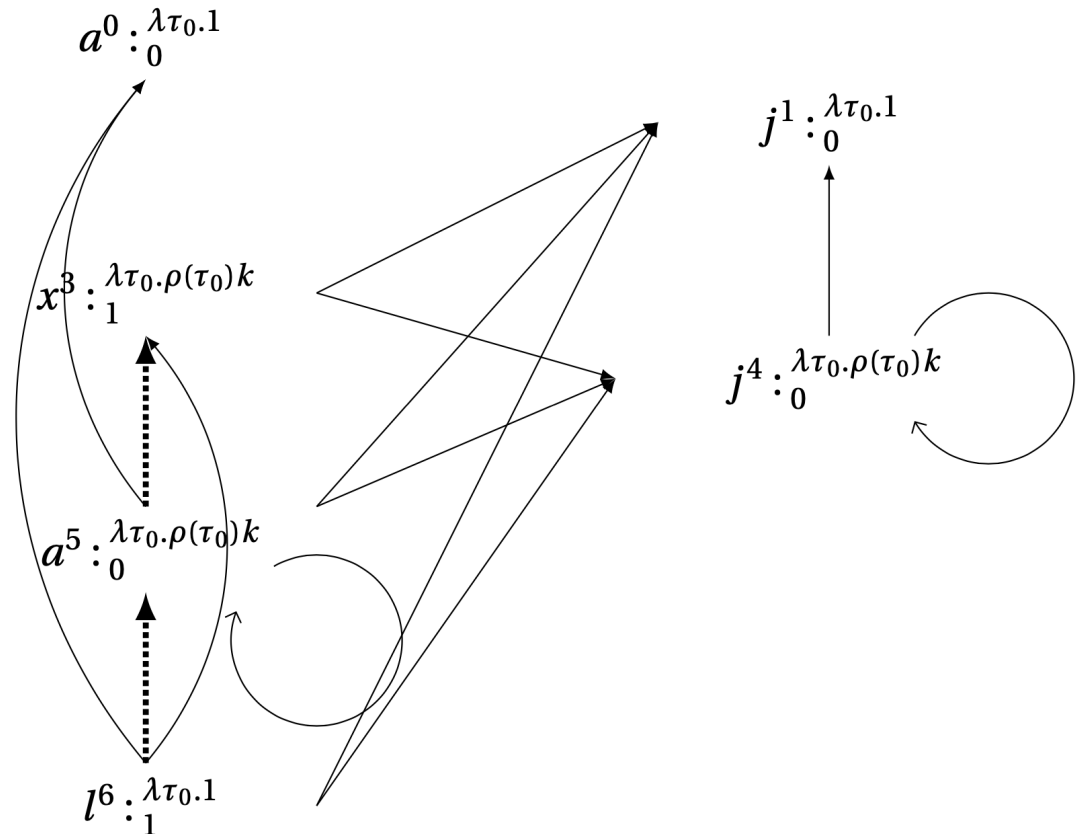
- Vertex $V(c)$: all the labeled variables in c ;
- Edge $E(c)$: *variable may-dependency* ;
- Weight $W(c)$: *weight* of each labeled variable x^l in c ;
- Query Annotation $Q(c)$: *query request*.

carrying the two sub-properties

```

twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query( $\mathcal{X}[j]$ )]3;
[j ← j - 1]4;
[a ← a + x]5);
[l ← query( $\mathcal{X}[k] * a$ )]6

```



→ Adaptivity as a "quantitative-reachability" hyper-property

$$G(c) = (V(c), E(c), W(c), Q(c))$$

Length of finite walk $\text{len}^q(k) : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$

a finite walk $k \in WALK(G(c))$

- vertices sequence: $(v_1, \dots, v_n), v_i \in V(c)$
- edges sequence: $(e_1, \dots, e_{n-1}), e_i = (v_i, v_{i+1}) \in E(c)$

the length of k:

$$\text{len}^q(k) = \lambda \tau_0. (\#v_i \text{ s.t. } v_i \in (v_1, \dots, v_n) \wedge Q(v_i) = 1)$$

→ Adaptivity as a "quantitative-reachability" hyper-property

Adaptivity $A(c) : \mathcal{T}_0^+(c) \rightarrow \mathbb{N}$

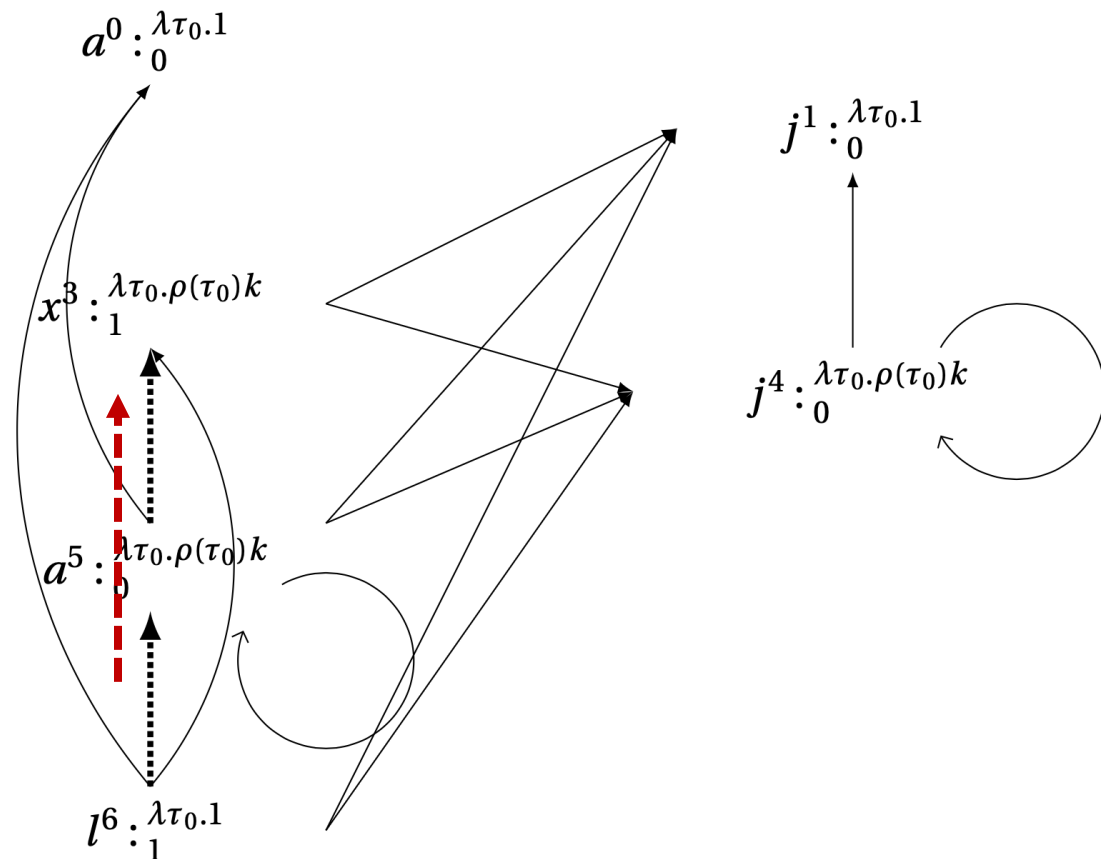
$$A(c) = \lambda \tau_0. \max \{ \text{len}^q(k)(\tau_0) \mid k \in \text{WALK}(G(c)) \}$$

length of the longest finite walk

```

twoRounds(k) :=
[a ← 0]0;
[j ← k]1;
while [(j > 0)]2 do
(
[x ← query( $\mathcal{X}[j]$ )]3;
[j ← j - 1]4;
[a ← a + x]5);
[l ← query( $\mathcal{X}[k] * a$ )]6

```



Program Analysis Algorithm for Adaptivity

- ➔ "Reachability" sub-property:
data dependency estimation by data & control-flow analysis
- ➔ "Quantitative" sub-property :
weight estimation by reachability-bound analysis
- ➔ Estimate "structure"
estimated dependency graph
- ➔ "Adaptivity"
compute upper bound by longest walk estimation algorithm

→ "Reachability": data-dependency analysis

data & control-flow and reaching definition analysis^[1]

$\text{flowsTo}(x^i, y^j, c) \in \{\text{true}, \text{false}\}$

$$\begin{aligned} \text{flowsTo}(x^i, y^j, [y \leftarrow e]^j) &\triangleq (x^i, y^j) \in \{(x^i, y^j) \mid x \in \text{FV}(e) \wedge x^i \in \text{RD}(l, [y \leftarrow e]^j)\} \\ \text{flowsTo}(x^i, y^j, [y \leftarrow \text{query}(\psi)]^j) &\triangleq (x^i, y^j) \in \{(x^i, y^j) \mid x \in \text{FV}(\psi) \wedge x^i \in \text{RD}(l, [y \leftarrow \text{query}(\psi)]^j)\} \end{aligned}$$

...

Soundness:

$$\forall c, x^i, y^j \in c. \text{DEP}(x^i, y^j, c) \Rightarrow \text{flowsTo}(x^i, y^j, c)$$

[1] Nielson F., H.R. Nielson; , C. Hankin (2005). Principles of Program Analysis. Springer. [ISBN 3-540-65410-0](https://doi.org/10.1007/978-3-540-65410-0).

→ "Quantitative" analysis[1]

$\hat{w} \in \text{Expr}(\mathbb{N} \cup \text{InputVar} \cup \{\infty\})$

```
twoRounds(k) :=
  [a ← 0]0;
  [j ← k]1;
  while [(j > 0)]2 do (
    [x ← query(X[j])]3;
    [j ← j - 1]4;
    [a ← a + x]5);
  [1 ← query(X[k] * a)]6
```

Soundness:

$\forall c \in \mathcal{C}, x^l \in \text{LV}(c), \tau_0 \in \mathcal{T}_0^+(c), \tau \in \mathcal{T}^+, v \in \mathbb{N}, l', (x^l, w) \in W_{\text{trace}}(c) .$

$(x^l, \hat{w}) \in W_{\text{est}}(c) \wedge (\langle \tau_0, \hat{w} \rangle \Downarrow_e v \implies w(\tau_0) \leq v)$

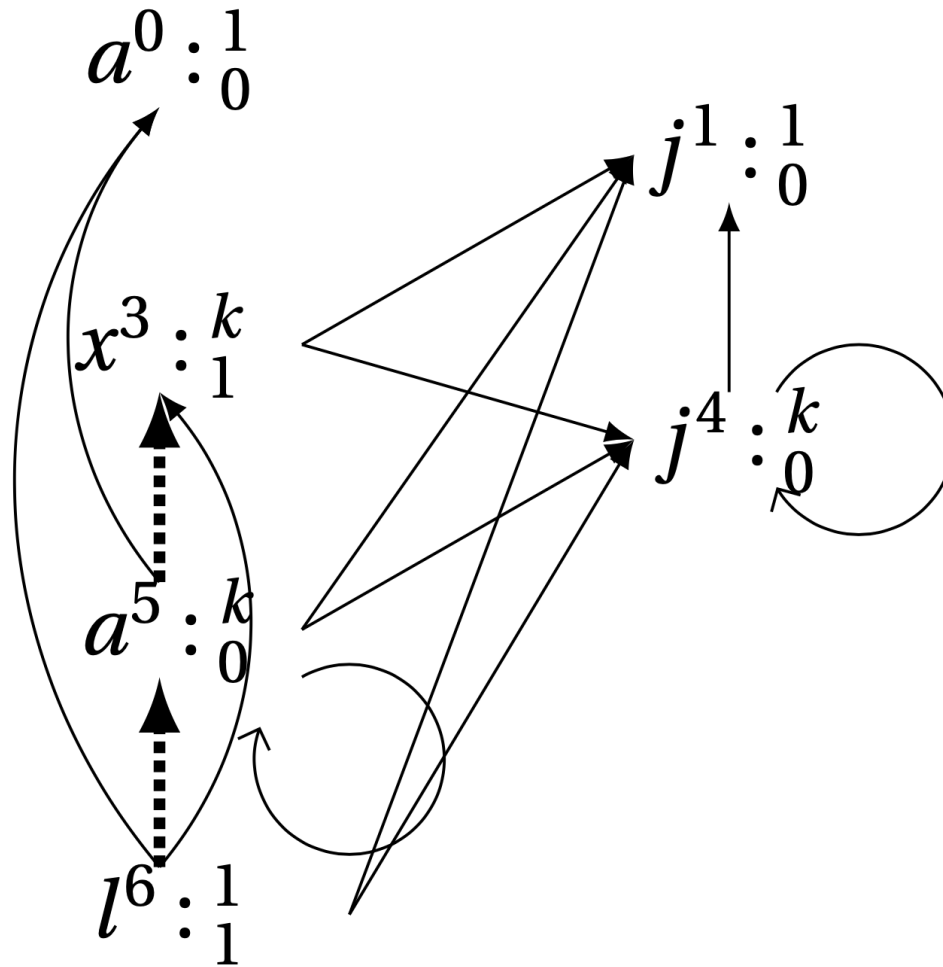
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

➔ Estimate "*semantics-based dependency graph*" as $\widehat{G}(c)$

$$\widehat{G}(c) = (V(c), \widehat{E}(c), \widehat{W}(c), Q(c))$$

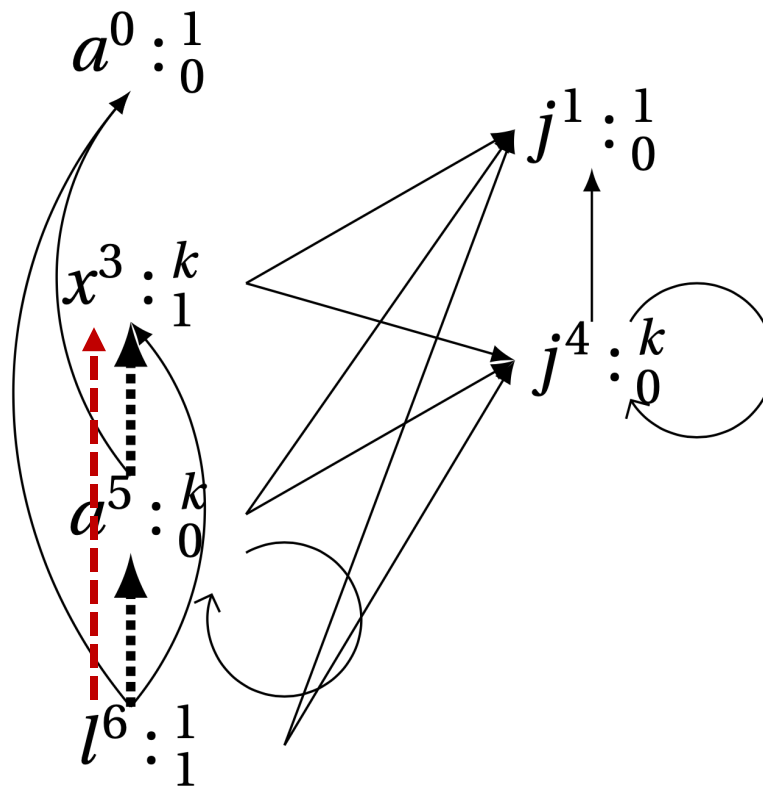
- Vertex $V(c)$: labeled variables in c
- Edge $\widehat{E}(c)$: *Estimated data-dependency relation*, $\text{flowsTo}(x^i, y^j, c)$
- Weight $\widehat{W}(c)$: *Estimated weight* for each labeled variable x^l in c ;
- Query Annotation $Q(c)$: labeled variables assigned by *query requests*.

➔ *"Estimated dependency graph"*



→ "Adaptivity" as the length of estimated longest walk

→ Estimate the upper bound on the program's "adaptivity"



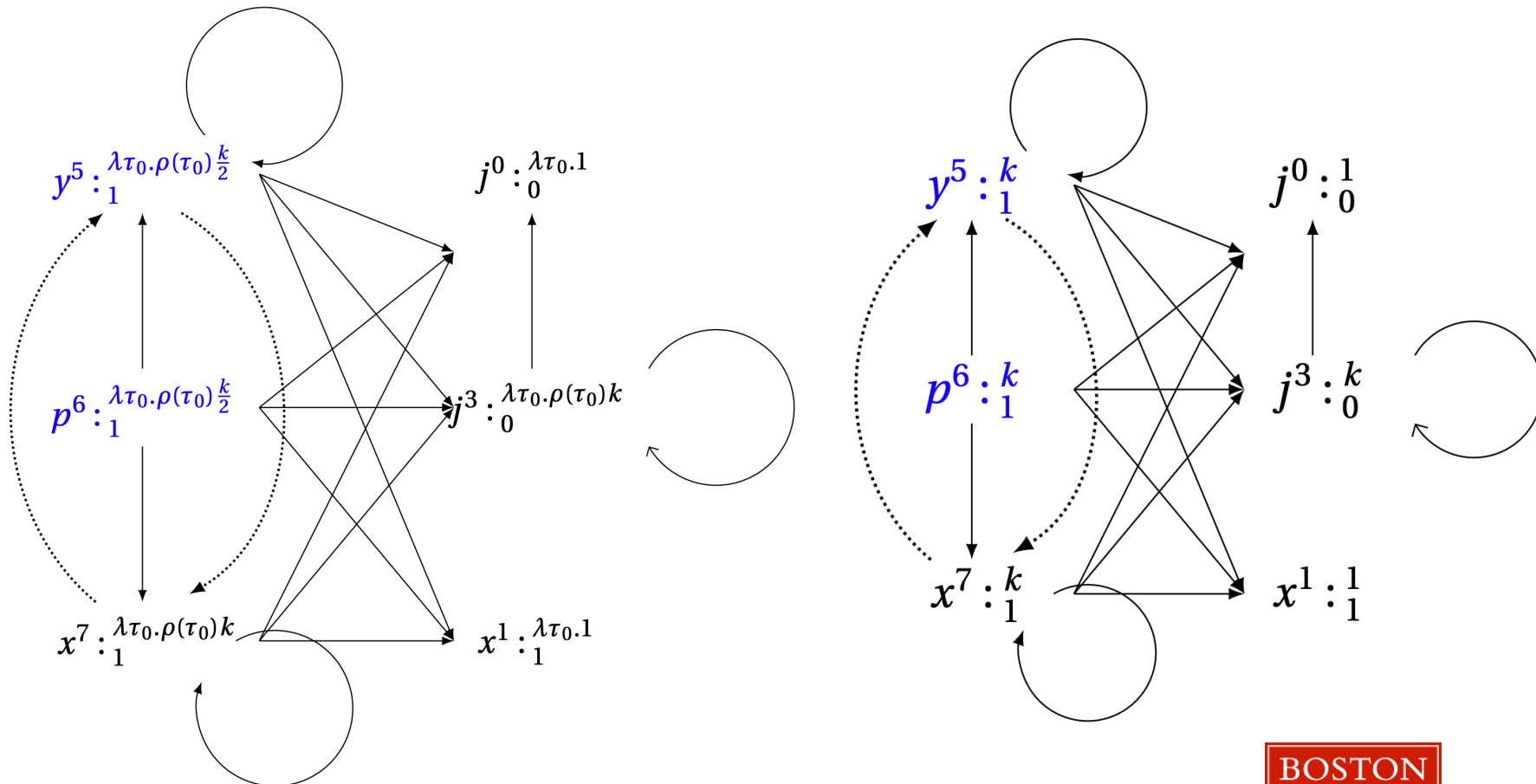
Limitation – imprecision of the *program analysis* algorithm

- The over-approximation in *multiple paths* loop bound

```
oddRounds(k) :=  
  [j ← 0] 0;  
  [x ← query( $\mathcal{X}$ [0])] 1;  
  while [(j > 0)] 2 do  
  (  
    [j ← j - 1] 3;  
    if ([j%2 == 0] 4,  
        [y ← x] 4; [p ← x] 5),  
        [x ← query( $\mathcal{X}$ [y])] 6  
    )  
  )
```

Limitation – imprecision of the program analysis algorithm

- The over-approximation in multiple paths loop bound

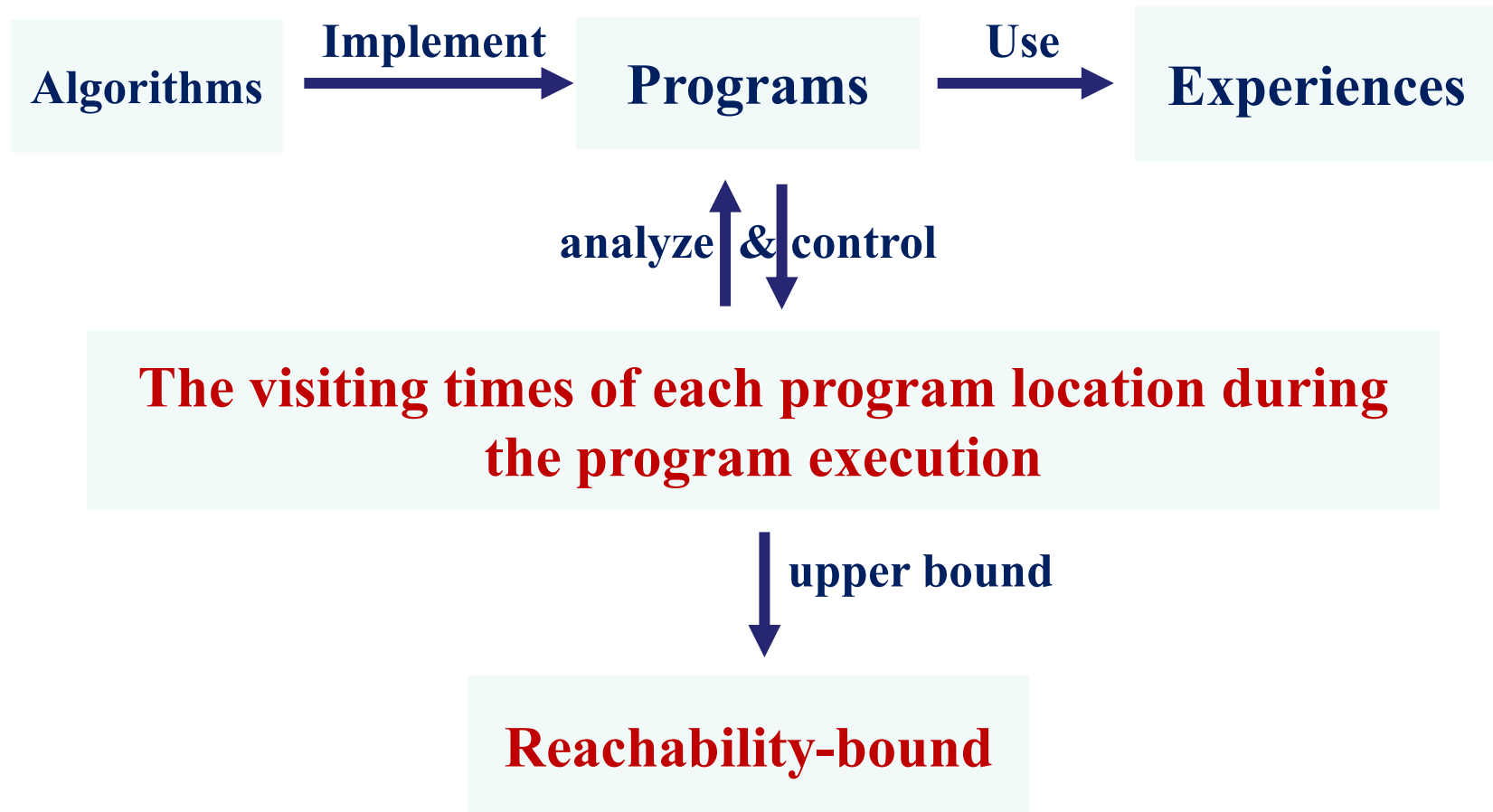


PART II :

Path-sensitive Reachability-bound Analysis

- Introduction:
 - Background
 - Motivation
- Language Model
- The Reachability-bound
- Path-sensitive Reachability-bound Algorithm

Introduction – two quantitative-reachability properties



Introduction – background

the number of times a given program location inside a procedure is visited during the program execution.

```

oddRounds(k) :=
[j ← 0]0;
[x ← query( $\mathcal{X}$ [0])]1;
while [(j > 0)]2 do
(
[j ← j - 1]3;
if ([j%2 == 0]4,
  [y ← x]4; [p ← x]5),
  [x ← query( $\mathcal{X}$ [y])]6 )
)

```

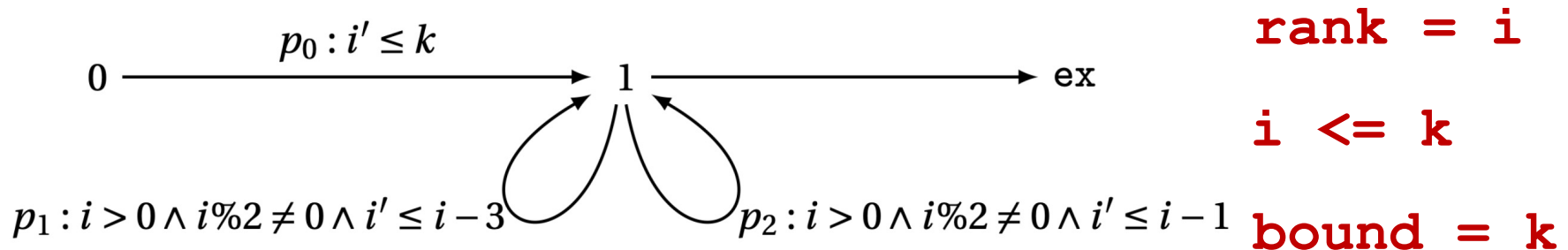
simplify

```

twoPaths(k) :=
[i ← k]0, → 1
while [i > 0]1 do → k/2+1
(
if ([i%2==0]2, → k/2
  then [i ← i - 1]3, → k/4
  else [i ← i - 3]4) → k/4
)

```

Introduction – technique challenges in the path-sensitivity amortized analysis with ranking function^[1]



```

twoPaths(k) :=
  [i ← k]0, → ?
  while [i > 0]1 do → k
    (if ([i%2==0]2, → k
      then [i ← i - 1]3, → k
      else [i ← i - 3]4) → k

```

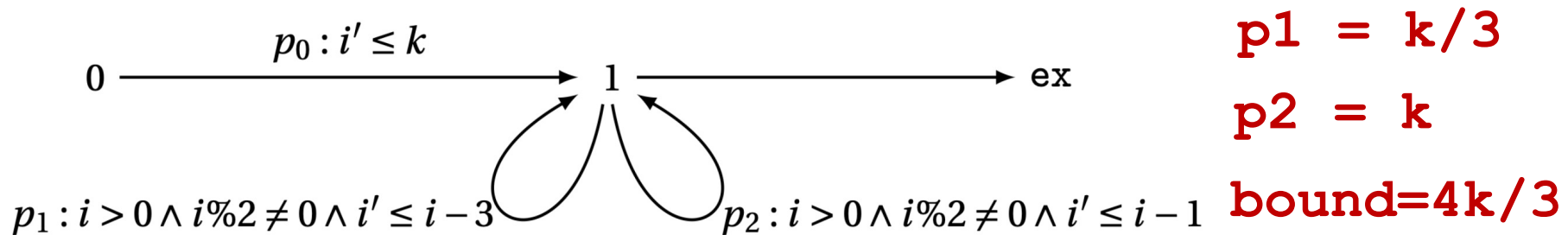
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in the path-sensitivity

loop summarization^[1,2,3]



```

twoPaths(k) :=
  [i ← k]0  $\xrightarrow{\quad}$  ?
  while [i > 0]1 do  $\xrightarrow{\quad}$   $4k/3 > k/2$ 
    (if ([i%2==0]2,  $\xrightarrow{\quad}$   $4k/3 > k/2$ 
      then [i ← i - 1]3,  $\xrightarrow{\quad}$   $k/3 > k/4$ 
      else [i ← i - 3]4)  $\xrightarrow{\quad}$   $k > k/4$ 

```

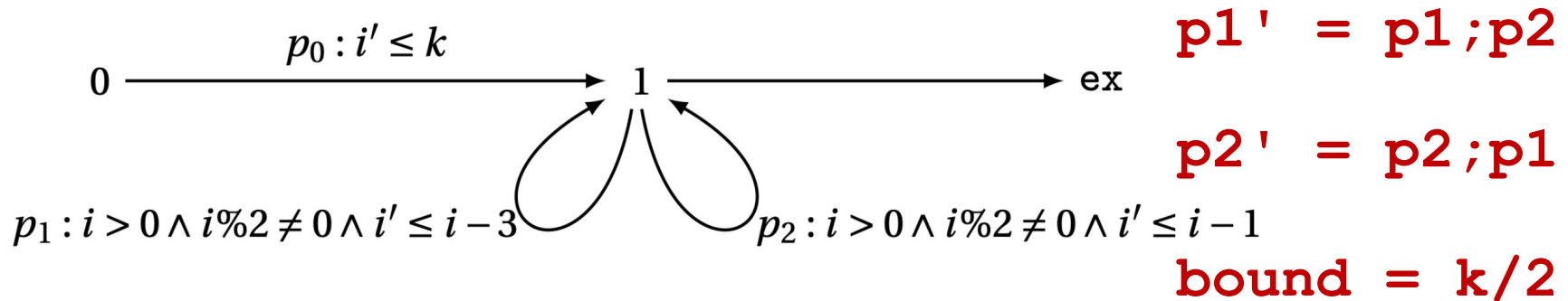
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in the path-sensitivity

loop path refinement^[3]



```

twoPaths(k) :=
  [i ← k]0 , → ?
  while [i > 0]1 do → k/2
    (if ([i%2==0]2 , → k/2
      then [i ← i - 1]3 , → k/2 > k/4
      else [i ← i - 3]4) → k/2 > k/4

```

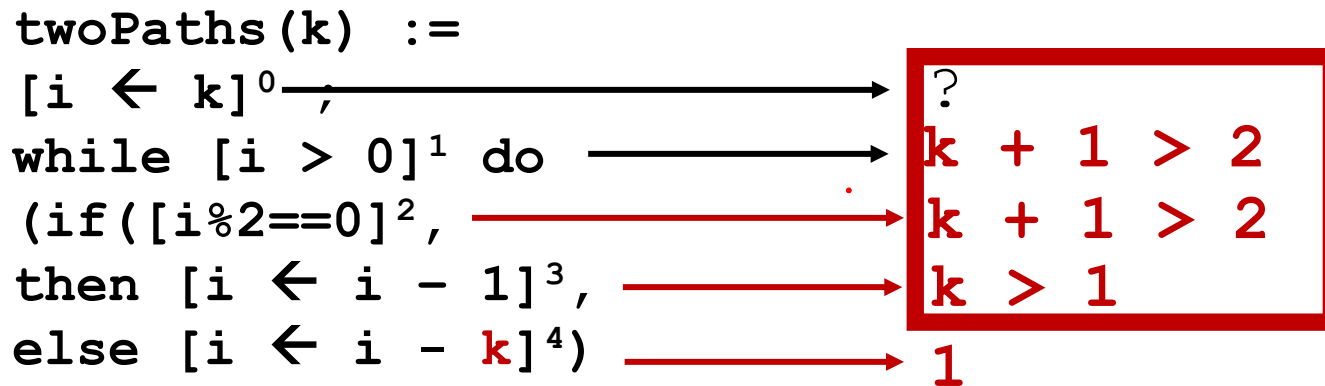
[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in the path-sensitivity

state-of-art^[1, 2, 3]



[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Introduction – technique challenges in nested loops

multiple loop paths and nested loops

`nestedWhileOdd(k)  $\triangleq$`

`[i  $\leftarrow$  k]0;`

`while [i > 0]1 do`

`( if ([i%2 = 0]2,`

`[i  $\leftarrow$  i - 1]3; [j  $\leftarrow$  k]4;`

`while [j > 0]5 do ([j  $\leftarrow$  j - 1]6),`

`[i  $\leftarrow$  i - 3]7); )`

expected bounds:

0 = 1;

1, 2 = k/2;

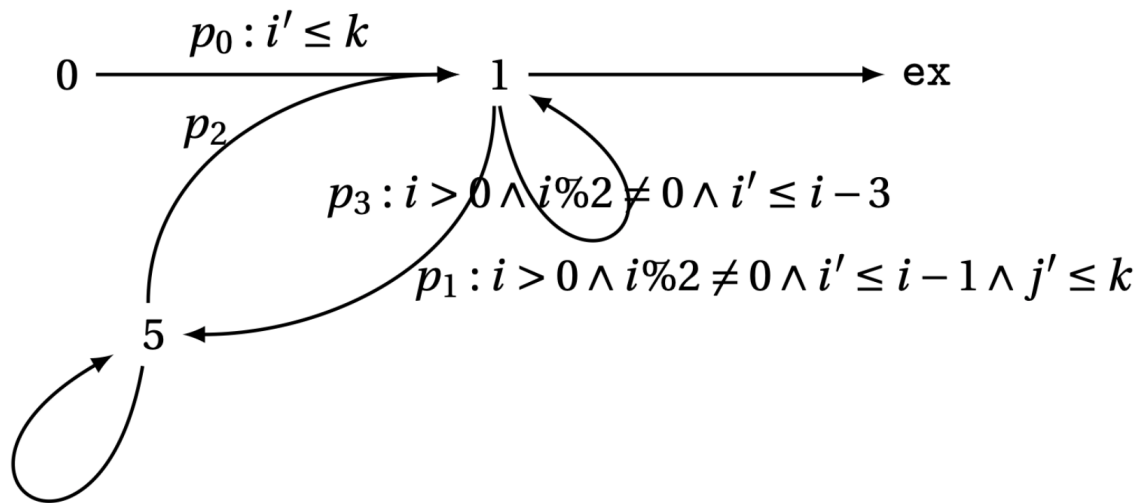
3, 4, 7 = k/4

5 = k/4 + k*k/4;

6 = k*k/4

Introduction – technique challenges in nested loops

state-of-art^[1,2,3]



$p_4 : i > 0 \wedge i \% 2 \neq 0 \wedge j \leq k \wedge j' \leq j - 1$

bound 1, 2, 3, 4, 7

$= \max(\text{bound}(p1+p2), \text{bound}(p3)) = k/2$

bound 5, 6 = bound(p4) $k*k/4$

[1] Moritz Sinn, Florian Zuleger, and Helmut Veith. Complexity and resource bound analysis of imperative programs using difference constraints. JAR 2017

[2] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

[3] Sumit Gulwani, Sagar Jain, and Eric Koskinen. Control-flow refinement and progress invariants for bound analysis. PLDI 2009

Language model – expression evaluation

$$\llbracket \cdot \rrbracket : \mathcal{A} \cup \mathcal{B} \rightarrow \mathcal{T}^{+\infty} \rightarrow \mathcal{V}$$

 $\llbracket \cdot \rrbracket : \text{Arithmetic Expression} \rightarrow \text{Trace} \rightarrow \text{Arithmetic Value}$

$$\begin{array}{c}
 \frac{}{\llbracket n \rrbracket(\tau) = n} \qquad \frac{\rho(\tau)x = v}{\llbracket x \rrbracket(\tau) = v} \qquad \frac{\llbracket a_1 \rrbracket(\tau) = v_1 \quad \llbracket a_2 \rrbracket(\tau) = v_2 \quad v_1 \oplus_a v_2 = v}{\llbracket a_1 \oplus_a a_2 \rrbracket(\tau) = v} \\
 \\
 \frac{\llbracket a \rrbracket(\tau) = v' \quad \log v' = v}{\llbracket \log a \rrbracket(\tau) = v} \qquad \frac{\llbracket a \rrbracket(\tau) = v' \quad \text{sign } v' = v}{\llbracket \text{sign } a \rrbracket(\tau) = v}
 \end{array}$$

The symbolic bound expression is a subset of the expressions, and we use the same rules and notation for evaluating the bound expression.

The reachability-bound – counter notations

$$\perp < n, \forall n \in \mathbb{N}$$

$$\textit{label counter} \text{ cnt} : \mathcal{T}^{+\infty} \rightarrow \mathcal{L} \rightarrow \mathbb{N} \cup \{\perp\}$$

$$\textit{label list counter} \text{ lcnt} : \mathcal{T}^{+\infty} \rightarrow \mathcal{P}(\mathcal{L}) \rightarrow \mathbb{N} \cup \{\perp\}$$

return $\perp$ for any label if the trace is ∞

The reachability-bound – definition^[1]

Definition 3 (Reachability-bound) *For a program c and a location $l \in \mathbf{L}(c)$ in c , a function $f(c, l) : \mathcal{T}_0^+(c) \rightarrow (\mathbb{N} \cup \{\infty\})$ is a reachability-bound for l if and only if*

$$\forall \tau_0 \in \mathcal{T}_0^+(c), c' \in \mathcal{C}, l, l' \in \mathbf{L}(c), \tau \in \mathcal{T}^{+\infty} . \\ \left(\langle c, \tau_0 \rangle \rightarrow^* \langle [\mathbf{skip}]^{l'}, \tau_{0++\tau} \rangle \vee \langle c, \tau_0 \rangle \uparrow^\infty \tau_{0++\tau} \right) \implies f(c, l)(\tau_0) \geq \mathbf{cnt}(\tau, l)$$

[1] Sumit Gulwani and Florian Zuleger. The reachability-bound problem. PLDI 2010

Path-sensitive reachability-bound algorithm

1. Generate An Abstract Transition Graph
2. ***Refine*** the multipaths of loops in Program
3. Estimate the ***ranking functions*** and bound invariant
4. Estimate the ***path reachability-bound*** for every ***path***
5. Estimate the ***loop reachability-bound*** w.r.t. ***path*** and ***outer loops***
6. Estimate the ***reachability-bound*** for every ***program point***

1. Abstract transition graph:

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

- **Vertex**: $\text{absV}(c) = L(c) \cup \{\text{ex}\}$

- **Edges**: $\hat{e} = (l, dc, l')$

command with label l' can execute right after the execution of the one with label l .

- Constraints: dc

- Difference constraints : $x, y \in \text{Var}; v \in \mathbb{N} \cup \text{Input} \cup \{\infty\}$

$$l \xrightarrow{x' \leq y+v} l' \quad l \xrightarrow{x' \leq v} l'$$

- Boolean expressions:

$$l \xrightarrow{b} l'$$

- $\top$

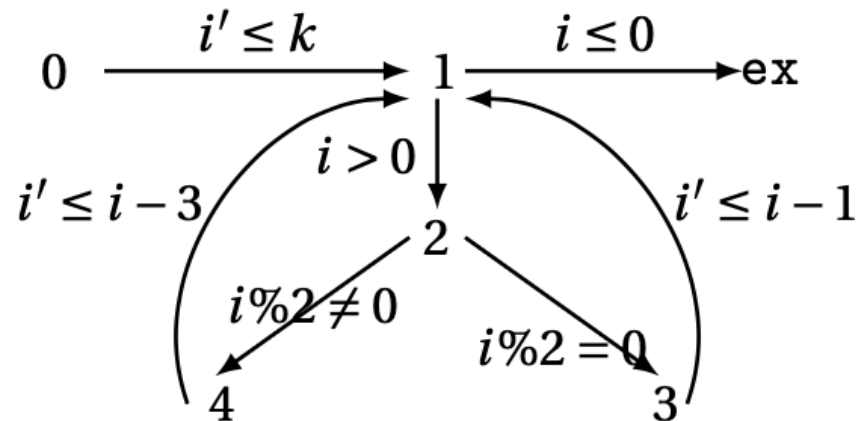
preserved for skip command, or commands that don't interfere any guard variable.

Example - abstract transition graph

```

twoPaths(k) :=
  [i ← k]0 ;
  while [i > 0]1 do
    (if([i%2==0]2,
      then [i ← i - 1]3,
      else [i ← i - 3]4)

```



***absG*(twoPaths(*k*))**

1. Abstract transition graph:

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

■ Path:

$$p \in \mathcal{PH}(\text{absG}(c))$$

■ path

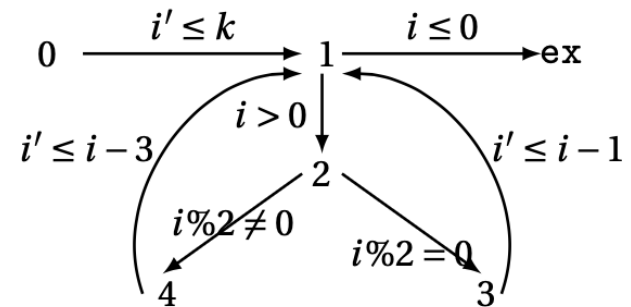
$1 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow \text{ex}$

■ cyclic path

$1 \rightarrow 2 \rightarrow 4 \rightarrow 1$

■ simple path

$0 \rightarrow 1, 1 \rightarrow \text{ex}, 1 \rightarrow 2 \rightarrow 3 \rightarrow 1, 0 \rightarrow 1 \rightarrow \text{ex}$



1. Abstract transition graph:

$$\text{absG}(c) \triangleq (\text{absV}(c), \text{absE}(c))$$

- *Simple transition path:*

$$\text{tp} \in \mathcal{PH}(\text{absG}(c))$$

simple and start- or end- point **has to** be either

- a loop header
- or program entry
- or program exit.

and no loop header except the start- or end- points.

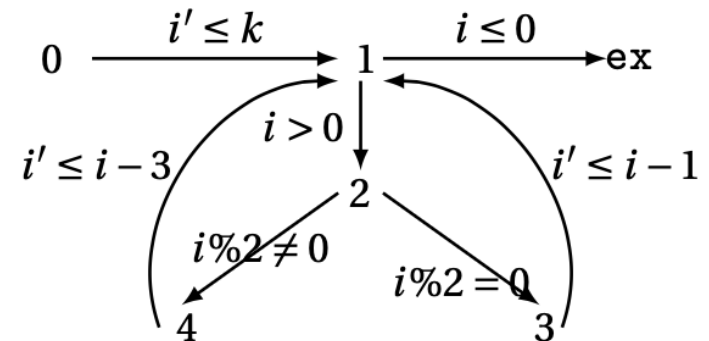
Example - simple transition path

- Simple transition path

$0 \rightarrow 1, 1 \rightarrow \text{ex}, 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

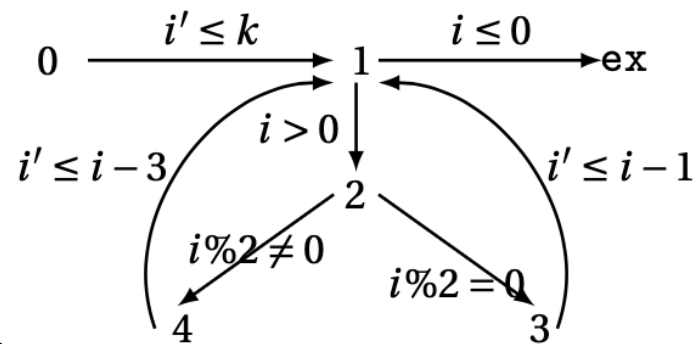
- Path that is simple but not a simple transition path

$0 \rightarrow 1 \rightarrow \text{ex}$



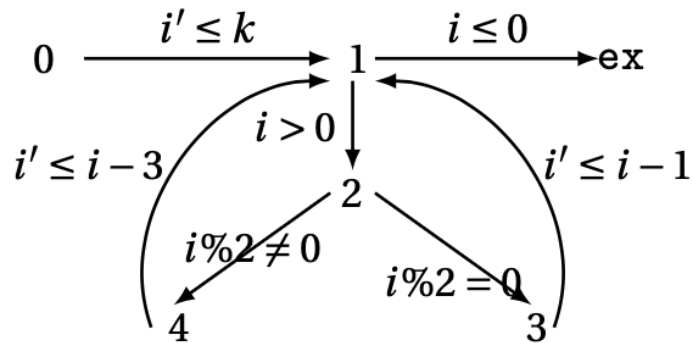
2. Program refinement

- Rewrite and compute new loop paths



- The rewritten program
 - `tp1; choose{repeat(tp1), repeat(tp2)}; tp3`
- New loop paths
 - `rp1 = 1 → 2 → 3 → 1; 1 → 2 → 4 → 1 = tp1; tp2`
 - `rp2 = 1 → 2 → 4 → 1; 1 → 2 → 3 → 1 = tp2; tp1`
- The refined program, *rp*
 - `rp = tp1; choose{repeat(rp1), repeat(rp2)}; tp3`

To know iteration times of the new loop paths.



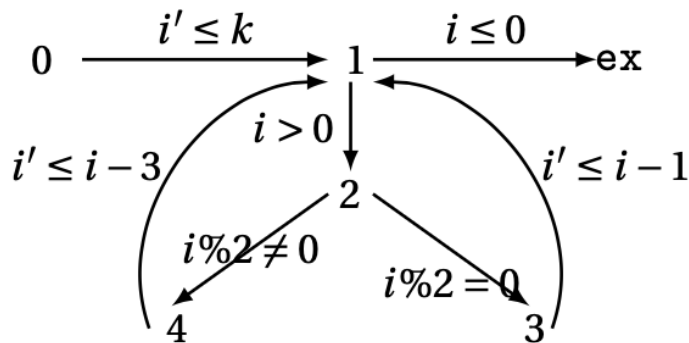
$$\frac{k}{4}$$

- $\text{rp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1$
- $\text{rp2} = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

3. Ranking function estimation

Ranks: $\text{rank}(\hat{e}, c) \in \text{Var} \cup \{\infty\}$

Ranking bound: $\text{VB}(x, c) \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\}) \subseteq \mathcal{A}$



4. Loop bound and local path reachability-bound

- Program states:

$\text{rInit}(\mathbf{rp}, \mathbf{c}), \text{rFinal}(\mathbf{rp}, \mathbf{c}), \text{rNext}(\mathbf{rp}, \mathbf{c}) \in \mathcal{P}(x = e),$

$x \in \mathit{Var}, e \in \mathit{Expr}(\mathbb{N} \cup \mathit{Input} \cup \{\infty\})$

- $\text{rInit}(\mathbf{rp}, \mathbf{c})$

initial value of the ranking functions before any visit of $\mathbf{rp}$,

- $\text{rNext}(\mathbf{rp}, \mathbf{c})$

how much $\mathbf{rp}$'s ranking functions are modified before the second visit of $\mathbf{rp}$

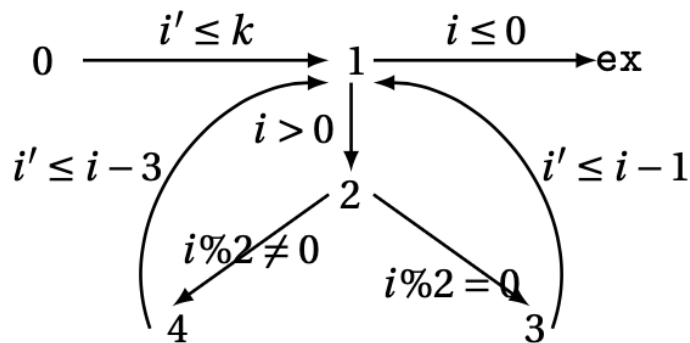
- $\text{rFinal}(\mathbf{rp}, \mathbf{c})$

minimum value of $\mathbf{rp}$'s ranking functions are after execution of $\mathbf{rp}$

4. Loop bound for new loop paths

$$\underline{BD(rp, c)} \in Expr(\mathbb{N} \cup Input \cup \{\infty\})$$

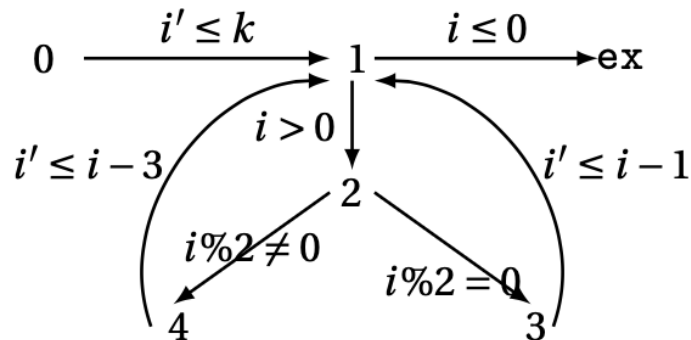
Example – loop bound for new loop paths



$$\max\left(\frac{k-0}{4}, \frac{k-0}{4}\right) = \frac{k}{4}$$

- $\text{rp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1; 1 \rightarrow 2 \rightarrow 4 \rightarrow 1 \Rightarrow \frac{k-0}{4}$
- $\text{rp2} = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1; 1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \Rightarrow \frac{k-0}{4}$

- To know iteration times of the simple transition paths.

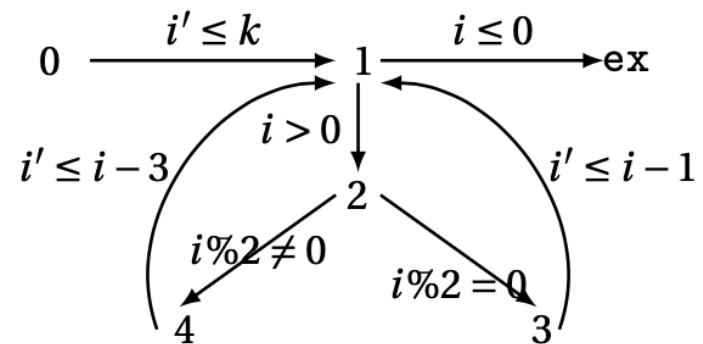


$$\frac{k}{4}$$

- $\text{tp1} = 1 \rightarrow 2 \rightarrow 3 \rightarrow 1,$
- $\text{tp2} = 1 \rightarrow 2 \rightarrow 4 \rightarrow 1$

4. Path reachability-bounds locally

$$\text{pathLocRB}(rp, tp) \in \text{Expr}(\mathbb{N} \cup \text{Input} \cup \{\infty\})$$



$\text{pathlocRB}(rp, tp1)$

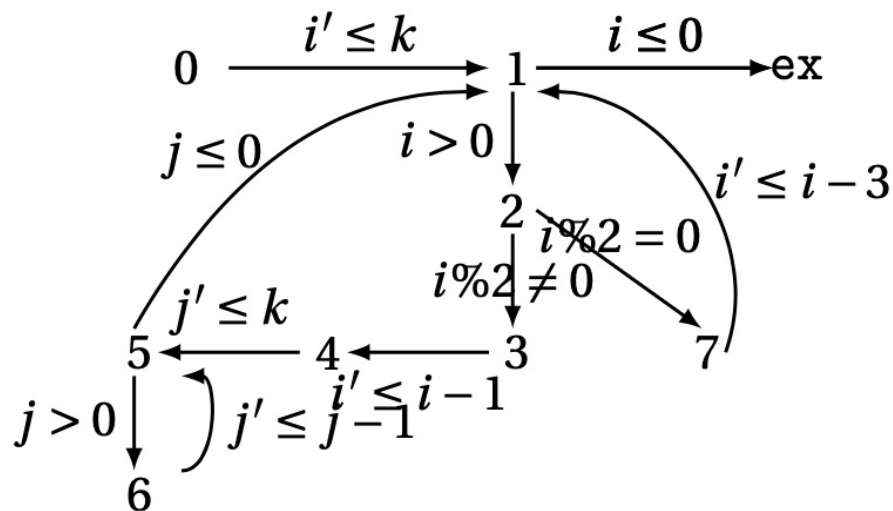
$$= \max \left\{ \text{pathlocRB}(rp1, tp1), \right. \\ \left. \text{pathlocRB}(rp2, tp1) \right\}$$

$$= \max \left\{ \text{pathlocRB}(tp1; tp2, tp1) * \text{BD}(rp1, c), \right. \\ \left. \text{pathlocRB}(tp1; tp2, tp1) * \text{BD}(rp1, c) \right\}$$

$$= \max \left\{ 1 * \frac{k}{4}, 1 * \frac{k}{4} \right\} = \frac{k}{4}$$

Not precise in case with nested loops.

Example with nested loops



$tp_0 = (0 \rightarrow 1)$
 $tp_1 = (1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5)$
 $tp_2 = (5 \rightarrow 6 \rightarrow 5)$
 $tp_3 = (1 \rightarrow 7 \rightarrow 2 \rightarrow 1)$
 $tp_4 = (1 \rightarrow ex)$
 $tp_5 = (5 \rightarrow 1)$

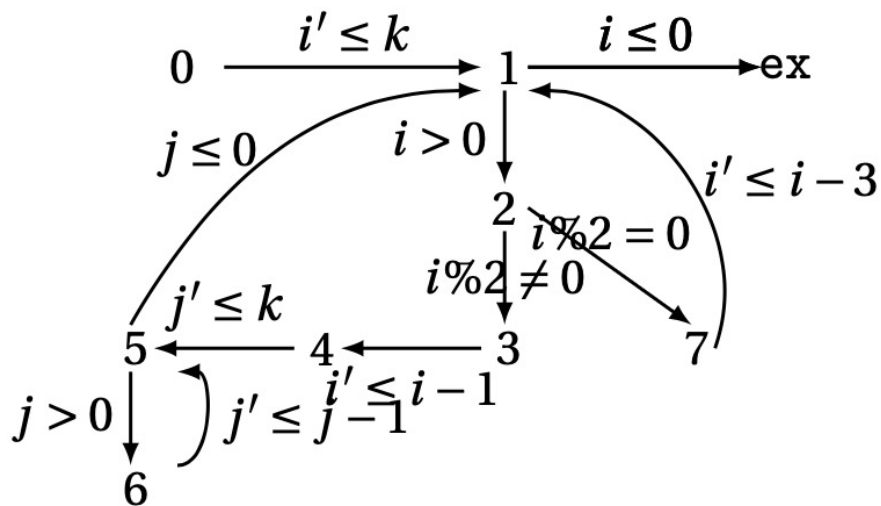
- New loop paths :
 - $rp1 = tp1; \text{repeat}(tp2); tp5; tp3$
 - $rp2 = tp3; tp1; \text{repeat}(tp2); tp5$
- The refined program
 - $rp = tp1; \text{choose}\{\text{repeat}(rp1), \text{repeat}(rp2)\}; tp4$

5. Loop reachability-bounds

$$\text{lpRB}(\mathbf{rp}', \mathbf{tp}) \in \mathbf{Expr}(\mathbb{N} \cup \mathbf{Input} \cup \{\infty\})$$

The upper bound on iteration numbers of the outer loop, such that, during these iterations, the inner loop is entered.

Example - loop reachability-bounds



■ $tp2 = 5 \rightarrow 6 \rightarrow 5$

outer loops:

$rp1 = tp1; \text{repeat}(tp2); tp5; tp3$

$rp2 = tp3; tp1; \text{repeat}(tp2); tp5$

$\text{loopRB}(rp1, tp2)$

$$= \max \left\{ \frac{(r_{\text{Inital}}(tp2) - r_{\text{Final}}(rp1))}{\max\{r_{\text{Next}}(rp1)\}} \right\}$$

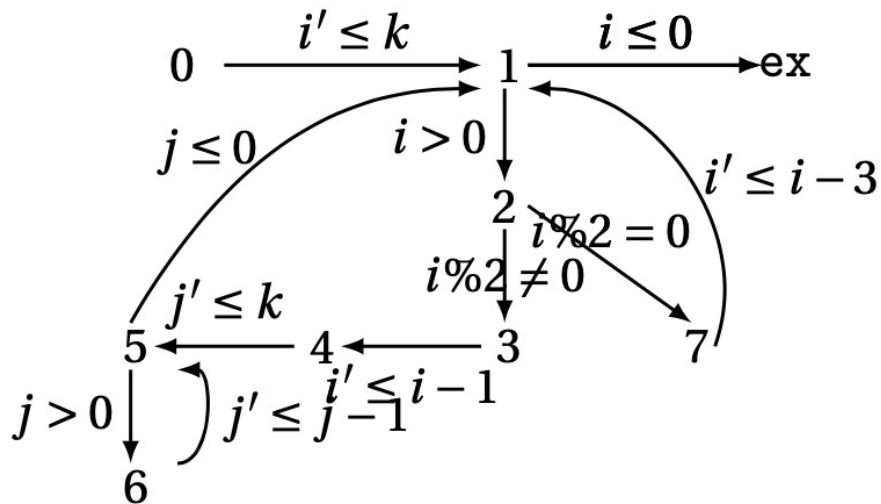
$$= \max \left\{ \frac{(k - 0)}{\max\{0, 4\}} \right\} = \frac{k}{4}$$

Path reachability-bound globally

$$\text{pathRB}(\mathbf{tp}, \mathbf{c}) \in \mathbf{Expr}(\mathbb{N} \cup \mathbf{Input} \cup \{\infty\})$$

the upper bound on the execution times of a simple transition path, $\mathbf{tp}$ when executing the program.

Example - path reachability-bound globally



■ $tp2 = 5 \rightarrow 6 \rightarrow 5$

outer loops:

$rp1 = tp1; \text{repeat}(tp2); tp5; tp3$

$rp2 = tp3; tp1; \text{repeat}(tp2); tp5$

$$\begin{aligned}
 & \text{pathRB}(rp, tp2) \\
 &= \max \left\{ \text{loopRB}(rp1, tp2), \text{loopRB}(rp2, tp2) \right\} * \text{pathRB}(\text{repeat}(tp2), tp2) \\
 &= \frac{k}{4} * \text{pathlocRB}(\text{repeat}(tp2), tp2) = \frac{k^2}{4}
 \end{aligned}$$

7. Reachability-bound for every location

$$\text{psRB}(\mathbf{tp}, \mathbf{c}) \in \mathbf{Expr}(N \cup \mathbf{Input} \cup \{\infty\})$$

$$\text{psRB}(l, c) = \sum \{\text{pathRB}(\mathbf{rp}, \mathbf{tp}, c) \mid \mathbf{tp} \in \mathbf{rp} \wedge l \in L_{\text{path}}(\mathbf{tp})\}$$

Soundness:

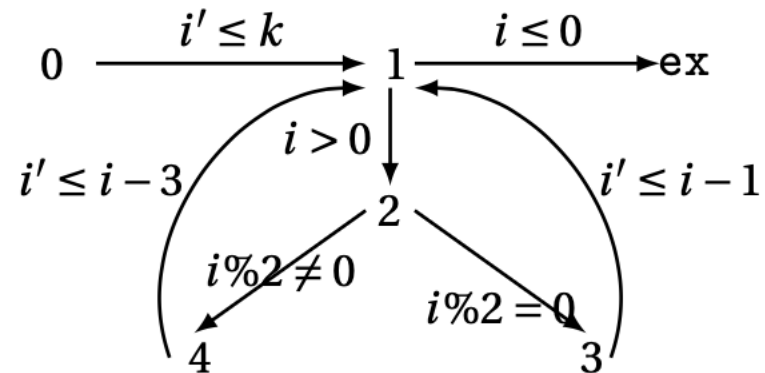
$\llbracket \text{psRB}(l, c) \rrbracket$ is a reachability-bound for program c at location l

Example - reachability-bound

```

twoPaths(k) :=
  [i ← k]0
  while [i > 0]1 do
    (if ([i%2==0]2,
      then [i ← i - 1]3,
      else [i ← i - 3]4)

```



The estimated visiting time for each program point

$$\text{psRB}(0) = \text{psRB}(\text{ex}) = 1 \quad \text{psRB}(3) = \text{psRB}(4) = \lfloor \frac{n}{4} \rfloor \quad \text{psRB}(1) = \text{psRB}(2) = \lfloor \frac{n}{2} \rfloor + 1$$

Example – reachability-bound

```

nestedWhileOdd( $k$ )  $\triangleq$ 
 $[i \leftarrow k]^0$ ;
while  $[i > 0]^1$  do
  (
    if  $([i \% 2 = 0]^2$ ,
       $[i \leftarrow i - 1]^3; [j \leftarrow k]^4$ ;
      while  $[j > 0]^5$  do  $([j \leftarrow j - 1]^6)$ ,
         $[i \leftarrow i - 3]^7$ );
    )
  )

```

accurate bounds:

$0 = 1;$

$1 = k/2 + 1;$

$2 = k/2;$

$3, 4, 7 = k/4$

$5 = k/4 + k*k/4;$

$6 = k*k/4$

Implementation and empirical evaluation

OCaml: Abstract Control Flow Graph

Python: Program refine, Ranking Estimation and Path-sensitive Reachability-bound algorithm

Benchmark	# P	M.P.L #	Nested Loop #	Bounded	Success Rate	Failed	Time Outs	Total Runtime
Loopus	110	53	52	98	89.1%	11	7	7min42sec
Loopus-C	23	20	20	20	86%	2	3	12min39sec
Icra	118	111	21	107	90.1%	11	1	4min48sec
Tianhan	37	2	3	37	100.0%	1	0	1min03sec

Implementation and empirical evaluation

Benchmarks: 270 from four benchmark sets

- 20 self-adopted programs
- 75.c programs from 7 benchmarks:

ABC: 15, C4B: 15, KoAT: 3, Loopus: 10, Rank10: 3, SPEED: 27, WTC: 37

- 37 challenging.c programs from the **LoopusJAR** [1]
- 37.java programs from **Tianhan**[2]
- 118.c programs from **Icra** [3]

[1] <https://forsyte.at/static/people/sinn/loopusJAR/index.html>

[2] <https://zenodo.org/record/5140586#.Y5pBoC-B1QI>

[3] <https://github.com/icra-team/icra>

Boston University Department of Computer Science

Implementation and empirical evaluation

Table 13.1: The Effectiveness Evaluation Table of psRB

Bench- marks	theory worst	(# p)	(# l)	reachability-bound (#1)						worst bound (# p)				
				1	n	n^2	n^3	n^4	∞	psRB	Loopus	Cofloco	SPEED	Benchmark (2021a)
Other Tools	$O(1)$	4	82	32	0	0	0	0	0	1	2	3	2	1
	$O(n)$	59	373	112	261	0	0	0	0	48	51	45	46	40
	$O(n^2)$	31	414	70	191	153	0	0	0	37	29	34	37	49
	$n\log(n)$	6	81	0	0	0	0	0	0	0	0	0	0	0
	$O(n^3)$	2	91	6	16	13	13	0	0	4	3	2	5	7
	$O(n^4)$	3	36	4	9	9	12	11	0	4	4	3	5	5
Loopus	$O(1)$	1	154	0	0	0	0	0	0	0	1	0	0	0
	$O(n)$	13	176	49	105	0	0	0	0	12	13	14	14	11
	$O(n^2)$	4	64	4	13	15	0	0	0	3	2	5	2	6
	$O(n^3)$	4	193	2	7	10	4	0	0	2	1	2	2	3
	$O(n^4)$	1	21	1	3	6	8	2	0	1	1	1	1	0
Icra	$O(1)$	4	93	41	0	0	0	0	0	1	3	2	2	0
	$\log(n)$	2	16	0	0	0	0	0	0	0	0	0	0	0
	$O(n)$	79	988	267	698	0	0	0	0	78	80	82	78	77
	$O(n^2)$	12	151	26	110	26	0	0	0	18	14	11	16	17
	$O(n^3)$	1	9	1	3	3	2	0	0	1	1	4	2	4
	∞	20	295	50	0	0	0	0	245	9	0	0	0	0
Tianhan	$O(n)$	35	188	68	120	0	0	0	0	35	35	35	35	35
	$O(n^2)$	2	12	4	8	4	0	0	0	2	2	2	2	2

Implementation and empirical evaluation

Bench- marks	theory worst	(# p)	(# l)	overall complexity bound	
				psRB	worst
Other Tools	$O(1)$	4	82	32	64
	$O(n)$	59	373	$112 + 261n$	$373n$
	$O(n^2)$	31	414	$70 + 191n + 153n^2$	$414n^2$
	$n \log(n)$	6	81	0	0
	$O(n^3)$	2	91	$6 + 16n + 13n^2 + 13n^3$	$48n^3$
	$O(n^4)$	3	36	$4 + 9n + 9n^2 + 12n^3 + 11n^4$	$45n^4$
Loopus	$O(1)$	1	154	0	-
	$O(n)$	13	176	$49 + 105n$	$162n$
	$O(n^2)$	4	64	$4 + 13n + 15n^2$	$32n^2$
	$O(n^3)$	4	193	$2 + 7n + 10n^2 + 4n^3$	$23n^3$
	$O(n^4)$	1	21	$1 + 3n + 6n^2 + 8n^3 + 2n^4$	$20n^4$
Icra	$O(1)$	4	93	6	64
	$\log(n)$	2	16	0	0
	$O(n)$	79	988	$267 + 698n$	$965n$
	$O(n^2)$	12	151	$26 + 110n + 26n^2$	$162n^2$
	$O(n^3)$	1	9	$1 + 3n + 3n^2 + 2n^3$	$9n^3$
	∞	20	295	50	∞
Tianhan	$O(n)$	35	188	$68 + 120n$	$188n$
	$O(n^2)$	2	12	$4 + 8n + 4n^2$	$16n^2$

Future Works

Accurate Adaptivity Analysis Framework

- Accurate adaptivity definition
- Accurate adaptivity estimation with "path-sensitive reachability-bound algorithm"

Efficient Reachability-bound Algorithm

- More scalable implementation
- Leverage the possible non-terminating in program refining and the ranking function estimating.